Foundations: LLM Inference Kernels

Foundations — a book for this repo

> A first-principles companion to the kernels and journey docs. Built so that after reading + drilling + walkthrough, you can re-implement the Phase 1 and Phase 2 work from memory and teach it cold.
>
> Five parts: 1. **What we're actually computing** (this part) 2. The GPU mental model 3. Decode attention, naive → fast 4. KV-cache compression 5. Cross-cutting lessons
>
> Each chapter has **checkpoint questions** inline. Don't continue past a checkpoint until you can answer it in your own words — the rest of the book builds on those answers.

Part 1 — What we're actually computing

This part has nothing to do with CUDA. It's about the operation we want the GPU to do. Get this right first, or the rest of the book is just syntax.

1.1 The big picture — where attention sits

A Large Language Model like Llama 3.1 8B is a tower of *transformer blocks*, plus an embedding at the bottom and a classifier at the top. Llama 3 8B has 32 blocks stacked on top of each other. Each block does two things to its input:

```
input → attention → residual add → MLP → residual add → output
                        + LayerNorm          + LayerNorm
```

For our purposes the only thing inside a block that matters is the **attention** layer. Each block has its own attention; they don't share weights. When the model "thinks" about token 47 of the input, that token's representation flows up through 32 attention computations before becoming a prediction for token 48.

The MLP and the LayerNorms are easy from a kernel standpoint — they're elementwise + matmul, fast on standard libraries. The two things that dominate inference cost are:

1. **Attention** in every layer. This is what makes long-context inference slow: it has to look at every prior token.
2. **The big matmuls** in the MLP and the QKV/output projections. With the model in fp16, these are weight-bandwidth-bound; with 4-bit weight quantization they get much faster.

This repo attacks both: **Phase 1** + **2** = **attention** + **its KV cache**, **Phase 3** = **quantized matmul**. Phase 4 is the integration that ties them together.

Why is attention hard? Because of two things: - It's **all-to-all** — every output token depends on every prior token. So work grows with sequence length. - Its memory cost is the *KV cache*, which can dominate VRAM at long context.

Phase 1 makes the attention compute fast. Phase 2 shrinks the KV cache.

> **Checkpoint 1.1** - Why do we care more about kernel speed for attention and matmul than for LayerNorm? - In a 32-block model, how many distinct attention computations happen per decoded token?

1.2 Attention from scratch — the mechanical view

Forget Q, K, V for a moment. Here is what attention *does*, in plain English:

> Given a sequence of tokens, for each token, compute a new representation that is a **weighted average of the representations of all the other tokens**, where the weights say "how relevant is this other token to me?"

That's it. The whole purpose of attention is to mix information across positions, weighted by relevance. Everything else is plumbing for that idea.

Now the plumbing.

Each token in the input has a `d`-dimensional representation. Call this representation `x`. We're going to compute three different "views" of each token:

- **Query** (`q`): "what am I looking for in the other tokens?"
- **Key** (`k`): "what do I represent, that other tokens might look for?"
- **Value** (`v`): "what information do I carry to share?"

These are three linear projections of the same input `x`:

```
q = W_q · x       (size d)
k = W_k · x       (size d)
v = W_v · x       (size d)
```

`W_q`, `W_k`, `W_v` are learned weight matrices. For each token we get a Q, a K, and a V. Stacked across the whole sequence:

- `Q` is shape `[seq_len, d]`
- `K` is shape `[seq_len, d]`

- `V` is shape `[seq_len, d]`

Now the actual attention computation. For token `i`'s output, we want a weighted average of *all* tokens' V vectors. The weights come from matching token `i`'s Q against every token's K:

```
score_i_j  =  q_i · k_j                    (one scalar per pair)
weight_i_j =  softmax(score_i_j over j)    (weights sum to 1 over
j)
output_i   =  Σ_j  weight_i_j · v_j        (weighted average of
V's)
```

The softmax turns scores into a probability distribution. Tokens with high scores get most of the weight; low-score tokens contribute almost nothing.

In matrix form across the whole sequence:

```
S  =  Q · K^T           shape [seq_len, seq_len]   (every-pair
scores)
P  =  softmax(S, dim=last)                          (rows sum to
1)
O  =  P · V                                          (output,
[seq_len, d])
```

Two more details that matter:

The 1/√d scaling. The dot products `q · k` have variance proportional to `d`. For large `d`, scores get huge, and softmax saturates (one weight goes to 1, the rest go to 0 — useless). The fix: divide by `√d` before softmax, so the variance stays roughly 1 regardless of `d`. This is the `softmax_scale = 1 / sqrt(head_dim)` you see everywhere.

Causal masking. When generating text, token `i` is only allowed to look at tokens `≤ i`. We can't let it cheat by looking at future tokens during training. The fix: set `S_i_j = −∞` for `j > i`, so `softmax` gives those positions weight 0. We'll come back to this.

A worked numerical example

Let's pick tiny numbers: `d = 4`, `seq_len = 3`. We have three tokens. Their Q, K, V (just made up):

```
Q = [[ 1,  0,  0,  0],     (token 0's query)
     [ 0,  1,  0,  0],     (token 1's)
     [ 0,  0,  1,  0]]     (token 2's)
```

```
K = [[ 1,  0,  0,  0],
     [ 0,  1,  0,  0],
     [ 0,  0,  1,  0]]

V = [[10, 20, 30, 40],     (token 0's value)
     [50, 60, 70, 80],
     [90,100,110,120]]
```

Compute `S = Q · K^T`:

```
S[0,0] = 1·1 + 0 + 0 + 0 = 1.0
S[0,1] = 1·0 + 0·1 + 0 + 0 = 0
S[0,2] = 0
... and so on
S  =  [[1, 0, 0],
       [0, 1, 0],
       [0, 0, 1]]
```

Apply softmax (over each row), with `d=4` so scale = `1/√4 = 0.5`. The scaled scores are `S/2 = diag(0.5)` and off-diagonal 0. Row 0's softmax:

```
softmax([0.5, 0, 0]) ≈ [0.51, 0.245, 0.245]
```

Note that even with a score of 0.5 vs 0, the high-score position only gets weight 0.51 — softmax with score ranges around 0 to 1 is *not very peaky*. That's the point of scaling.

Then `O = P · V`:

```
O[0] = 0.51·[10,20,30,40] + 0.245·[50,60,70,80] +
0.245·[90,100,110,120]
     = [5.1, 10.2, 15.3, 20.4]
     + [12.25, 14.7, 17.15, 19.6]
     + [22.05, 24.5, 26.95, 29.4]
     = [39.4, 49.4, 59.4, 69.4]
```

So token 0's new representation is `[39.4, 49.4, 59.4, 69.4]` — a mix weighted ~half towards itself, ~quarter each towards the others.

The mechanical takeaway: **attention is a soft, weighted average of V's, indexed by Q-against-K similarity.**

> **Checkpoint 1.2** - Without looking, write the attention formula `O = ...` in terms of `Q, K, V, d`. - Why do we divide by `√d` before softmax? What goes wrong if we don't? - If `Q` and `K` are identical and orthonormal across tokens, what is `softmax(Q·K^T / √d)` — close to identity or close to uniform?

1.3 Multi-head attention

One pass of attention as described above is *one head*. Real models do multi-head attention: split the `d`-dimensional space into `n_heads` chunks of `head_dim = d / n_heads`, run attention independently in each chunk, then concatenate.

The motivation: each head can learn a different "pattern of attention" — one might focus on syntactic relations, another on long-distance co-references, another on local n-grams. Empirically this works much better than a single fat attention computation.

In Llama 3 8B, the QKV projections produce vectors of dimension `n_heads × head_dim = 32 × 128 = 4096`. We reshape into `(n_heads, head_dim) = (32, 128)` and run attention independently in each of the 32 heads. The outputs `[32, head_dim]` get concatenated back to a `4096`-dim vector and projected to the original `d` via `W_o`.

Tensor shapes during a *training-style* forward pass (where everything fits in memory):

```
input       x : [batch, seq_len, d]                    d = 4096

q,k,v projections produce:
            q : [batch, seq_len, n_heads, head_dim]   = [b, s, 32,
128]
            k : [batch, seq_len, n_heads, head_dim]
            v : [batch, seq_len, n_heads, head_dim]

transpose to: [batch, n_heads, seq_len, head_dim]    (heads outer
for compute)

per-head:
            s = q · k^T / √head_dim   shape [b, n_heads, seq_len,
seq_len]
            p = softmax(s + mask)
            o = p · v                  shape [b, n_heads, seq_len,
```

```
head_dim]

transpose back, reshape, project:
            out = W_o · concat(o per head)            [batch,
seq_len, d]
```

The kernels we write live at the *per-head* layer, parallelised across the `n_heads` dimension. Each thread block handles one (batch, head) pair. That's the grid you saw in `grid(batch, n_heads)` in v0 → v3.

> **Checkpoint 1.3** - For Llama 3 8B with `d=4096, n_heads=32`, what is `head_dim`? - In multi-head attention, do the different heads share weights? Share inputs? Share outputs? - If we have batch=8, n_heads=32, seqlen=4096, head_dim=128, how big is `Q` (in fp16)?

1.4 GQA — Grouped Query Attention

In a vanilla transformer, each head has its own Q, K, V projections. So the K and V tensors have shape `[batch, n_heads, seq_len, head_dim]` — one per head, just like Q.

For inference, we have to *cache* K and V across decode steps (more on this in §1.6). That cache scales with `n_heads`. For Llama 3 8B with 32 heads × 32 layers, the cache gets large fast.

GQA cuts the cost: instead of one K, V pair per query head, **multiple query heads share the same K, V pair**. Llama 3 8B uses `n_heads = 32` query heads but only `n_kv_heads = 8` KV heads. Each KV head is shared by `n_heads / n_kv_heads = 4` query heads.

So the shapes diverge:

```
Q : [batch, n_heads,    seq_len, head_dim]   = [b, 32, s, 128]
K : [batch, n_kv_heads, seq_len, head_dim]   = [b,  8, s, 128]
(4× smaller)
V : [batch, n_kv_heads, seq_len, head_dim]   = [b,  8, s, 128]
```

For computation, we logically expand K and V back to 32 heads by repeating each KV head 4 times (this is `repeat_kv` in HF transformers source). But for *storage*, K and V keep the smaller `n_kv_heads` dimension — that's where GQA's memory savings come from.

In our kernels, we don't physically expand. We use an **index map**: when computing attention for query head `h`, look up K, V from kv head `kv(h) = h / (n_heads / n_kv_heads)`. For Llama 3 8B that's `kv(h) = h / 4`. So:

- query heads 0, 1, 2, 3 → kv head 0
- query heads 4, 5, 6, 7 → kv head 1
- …
- query heads 28, 29, 30, 31 → kv head 7

This is the `kv_head_idx = head_idx / (n_heads / n_kv_heads)` line in every kernel we wrote.

> **Checkpoint 1.4** - In Llama 3 8B, how many bytes does the K tensor take per token (one layer, fp16)? Hint: `n_kv_heads × head_dim × 2`. - For query head 17, which kv head does it index into? - GQA saves memory; does it cost anything? (Hint: think about model quality, not compute.)

1.5 Prefill vs decode — the M=1 problem

So far we've described attention as if all `seq_len` tokens are available at once and processed in parallel. That's how *training* works, and also how the **prefill** phase of inference works: when the user gives a prompt of, say, 512 tokens, the model processes all 512 in one shot.

Then comes **decode**: the model generates new tokens one at a time. For decode step `t`, the input is *just one new token's representation*. The model needs to produce a Q for that one new token, attend it against all *previously seen* K and V (the cache), and output one new representation.

Decode attention shape:

```
Q : [batch, n_heads,    1,         head_dim]   ← only one position
K : [batch, n_kv_heads, seqlen_kv, head_dim]   ← all prior
positions
V : [batch, n_kv_heads, seqlen_kv, head_dim]   ← all prior
positions
out : [batch, n_heads,  1,         head_dim]
```

`seqlen_kv` is the current context length — the number of tokens already processed. It grows by one with every decoded token.

This `Q` shape — a single query position — is what we mean by the **M = 1 problem**. Tensor Cores on modern GPUs are designed for matrix multiplies where M, N, K are all reasonably large (typical Tensor Core shape: M=N=K=16). When M=1, you're doing a `(1 × d) · (d × seq_len)` matmul. The GPU can do this, but most of the silicon dedicated to matmul is wasted — you're really doing a *gemv*, a matrix-vector product, not a matrix-matrix product.

That's the core reason decode attention is harder than prefill: - Prefill is **compute-bound for long prompts** and maps well to FlashAttention-2. - Decode is **memory-bound** in the limit — read the entire KV cache, do tiny per-position FLOPs, write one output. Memory bandwidth, not Tensor Core throughput, is the ceiling. (Or so the theory goes — Phase 1 will show it's more nuanced for our specific workload.)

The split between prefill and decode is why this repo focuses on a **decode** kernel for Phase 1. Decode dominates chat serving wall-clock (one token at a time × hundreds of generated tokens). Prefill happens once per request.

> **Checkpoint 1.5** - In decode, what is the shape of `Q`? What is `seqlen_kv` and what determines its value? - Why is decode harder for Tensor Cores than prefill? - What's the wall-clock split between prefill and decode for a chat request that takes 512 input tokens and generates 256 tokens? (You need a rough mental model — don't compute exactly.)

1.6 The KV cache — what we are caching and why

Naively, every decode step would re-read every prior token's representation from the start, project it through the layer's QKV weights, and use those K and V values. That re-projection is **redundant** because the layer's K and V for prior tokens are *fixed* once computed — they don't depend on the current decoding step.

So we **cache K and V** across decode steps. Each layer maintains a per-batch buffer of all K and V values for all previously seen positions, and just *appends* the new token's K, V to it each step.

That cache is the **KV cache**, and it has the shapes we've been seeing:

```
Per layer, per batch slot:
K : [n_kv_heads, max_seqlen, head_dim]   fp16
V : [n_kv_heads, max_seqlen, head_dim]   fp16
```

Why not cache Q? Q changes every step (it's the projection of the *new* token's input), and we only use the current step's Q for the current step's attention. So Q doesn't accumulate.

The 64 GB problem

The cache size grows linearly with sequence length AND with batch size. For Llama 3.1 8B:

```
bytes_per_token_per_layer = 2 (K, V) · n_kv_heads · head_dim · sizeof(fp16)
                          = 2 · 8 · 128 · 2
                          = 4096 bytes per token per layer
```

Across all `n_layers = 32` layers, that's `4096 · 32 = 131,072 bytes per token = 128 KiB per token` per batch slot. Wait — the design doc quotes 256 KB/token; let me redo the multiplication carefully:

```
2 (K and V) · 32 layers · 8 kv_heads · 128 head_dim · 2 bytes (fp16)
= 2 · 32 · 8 · 128 · 2
= 131,072 bytes = 128 KiB per token
```

(The docs/02 number of 256 KB/token uses `n_kv_heads=8` but counts each of K and V as `n_kv_heads · head_dim · 2 = 2048 bytes`, then `2 · 32 layers · 2048 = 131072 bytes` — same answer. The "256 KB" in docs/02 is slightly off; trust the calculation here.)

Now scale it up. For a chat-serving workload of `batch=32` users, `seqlen=8192` tokens each:

```
total_kv_bytes = batch · seqlen · 128 KiB
               = 32 · 8192 · 131072
               = ~34 GiB
```

A 24 GB RTX 4090 doesn't fit that, never mind 16 users. **KV cache, not weights, is what caps batch size and context length on consumer-class hardware.**

This is the constraint Phase 2 attacks. INT8 KV cuts this in half. INT4 KIVI cuts it to a quarter.

KV cache and bandwidth

There's a second cost to the KV cache beyond memory: every decode step reads **the entire** K and V from HBM into compute. For one (batch, head) at `seqlen_kv = 4096`, fp16:

```
K bytes per (batch, head) = seqlen_kv · head_dim · 2 = 4096 · 128
· 2 = 1 MiB
V bytes per (batch, head) = same = 1 MiB
total per (batch, head) = 2 MiB
total for batch=8 × n_heads=32 (Q heads): 256 · 2 MiB = 512 MiB
read per decode step
```

GQA reduces this — with `n_kv_heads=8`, each kv head's K, V is shared by 4 query heads, so unique reads are `8 · 8 · 1 MiB · 2 = 128 MiB`, not 512 MiB (modulo L2 cache behaviour). Still: every decode step is moving a sizable chunk of data from HBM. **Bandwidth is the second constraint** the KV cache imposes.

INT8 KV halves the bytes-per-element. INT4 quarters. That's the *memory bandwidth* benefit of compression, separate from the *memory capacity* benefit.

> **Checkpoint 1.6** - Why do we cache K and V but not Q? - At `batch=16, seqlen=4096`, how big is the fp16 KV cache for Llama 3 8B? Does it fit in 24 GB alongside the 16 GB of weights? - If we go from fp16 KV to INT8 KV, the memory savings are obvious. What's the *bandwidth* savings during decode?

1.7 What does a fast decode kernel look like, then?

Tying it together. A *single decode step* for one (batch, query-head) pair looks like this:

```
Inputs:
  q     : [head_dim]                    fp16    (one Q vector)
  K     : [seqlen_kv, head_dim]         fp16    (cached K for this
kv head)
  V     : [seqlen_kv, head_dim]         fp16    (cached V for this
kv head)
  scale : scalar  (1 / √head_dim)
```

```
Compute:
  s[j]    = scale · dot(q, K[j, :])         for j in
0..seqlen_kv-1
  m       = max(s)
  p[j]    = exp(s[j] − m) / Σ_j exp(s[j] − m)
  out[d]  = Σ_j p[j] · V[j, d]              for d in 0..head_dim-1

Output:
  out   : [head_dim]                    fp16
```

That's the entire decode attention for one query head, one batch slot, at one decode step. To get the full per-token output of the model:

- For each of `n_layers = 32` layers,
  - For each of `n_heads = 32` query heads,
    - Run the above. Use the index map `kv(h) = h/4` to pick the right KV head.
- Concatenate the 32 head outputs into one `4096`-dim vector and apply `W_o`.
- Add the MLP block.
- Add the LayerNorm.

The attention kernel we write does **the inner loop** — one (batch, head) pair's decode attention. It runs once per (batch, head) per layer per decode step. Latency of this kernel multiplies by `n_layers · n_heads` × decode steps to give wall-clock cost.

What success looks like

For a **kernel-level** win: - **Latency**: as fast as PyTorch SDPA. SDPA dispatches to FlashAttention/cuDNN — the state of the art. On our reference workload, SDPA hits 1.36 ms; our v3 hits 0.713 ms — 1.91× faster. - **Bandwidth**: meaningful fraction of HBM peak. Our v3 hits 189 GB/s of 1008 GB/s peak (19%). Not at peak, but well into useful range.

For a **memory** win: - **KV bytes per token** as small as quality allows. fp16 is 4096 B; INT8 is ~2080 B (incl. scales); INT4 KIVI is ~1080 B. We measured perplexity on WikiText-2 to confirm quality stays within budget.

For a **system** win: - Both kernels integrated into a real LM serving stack, so the microbench gains translate to user-facing tokens/sec. That's Phase 4.

> **Checkpoint 1.7** - Sketch the decode attention computation in 5 lines, using `q, K, V, scale` and a single output `out`. (You should be able to write this from memory by now.) - In our project, what wraps the kernel into per-layer, per-head form so a full decode step happens? (Hint: nothing yet for Phase 1/2 — we benchmark the kernel in isolation. Phase 4 integrates it.) - How many times per decoded token does the attention kernel run for Llama 3 8B with batch=8?

1.8 Summary of Part 1

You should now be comfortable with:

- **Attention** as a softly-weighted average of V's, indexed by Q-K similarity. The formula `O = softmax(Q · K^T / √d) · V`.
- **Multi-head** attention as `n_heads` independent attention computations on `head_dim = d / n_heads`-sized chunks.
- **GQA** as having `n_kv_heads < n_heads`, with the index map `kv(h) = h / (n_heads / n_kv_heads)`.
- **Prefill vs decode**: prefill is many queries in parallel, compute-bound; decode is one query against a growing KV cache, memory-bound, the "M = 1" problem.
- **KV cache**: cache K and V so we don't recompute them every step; size grows with batch × context; this is what caps serving throughput on consumer GPUs.
- **What the kernel does**: per (batch, head), score against all kv positions, softmax, weighted sum of V's. Returns one `[head_dim]` vector.

Everything from here on is about making this computation *fast*. Part 2 covers the GPU mental model: what hardware we have to work with, and how performance is actually limited.

1.9 Glossary (for quick reference later in the book)

Term	Meaning
`d` (also "d_model")	Hidden size of the model (Llama 3 8B: 4096).
`n_heads`	

Term	Meaning
	Number of attention heads per layer (Llama 3 8B: 32).
`n_kv_heads`	Number of *KV* heads under GQA (Llama 3 8B: 8).
`head_dim`	`d / n_heads` (Llama 3 8B: 128).
`seq_len`, `seqlen_kv`	Number of tokens in context.
`Q`, `K`, `V`	Query, Key, Value tensors.
`Q · K^T`	The "score" matrix.
`softmax_scale`	`1 / √head_dim`. Stabilises softmax.
Prefill	Initial pass over the prompt (many query positions).
Decode	One generated token at a time (one query position).
KV cache	Stored K, V from prior decode steps.
GQA	Grouped Query Attention — fewer KV heads than Q heads.
HBM	High Bandwidth Memory — the GPU's main DRAM.

Ready for Part 2? Part 2 (the GPU mental model) is where we build the *performance* intuition: how threads/warps/blocks map to hardware, the memory hierarchy, what __syncthreads costs, and the diagnostic framework (bandwidth-bound vs compute-bound vs dependency-chain-bound) that runs through every kernel we wrote.

Before continuing, try to answer at least the §1.6 and §1.7 checkpoints without looking. Those two are load-bearing for the rest of the book.

Part 2 — The GPU mental model

Part 1 told you *what* the kernel computes. Part 2 tells you *how the hardware will run it* and, crucially, *what will limit how fast*. This is the part of the book that you'll come back to most — it's the toolkit for reading any kernel and knowing where the time is going.

2.1 Why a mental model?

When we write a CUDA kernel, what we're really doing is *describing a parallel computation to the hardware*. The hardware then has a lot of discretion in *how* it runs it — which threads execute when, which loads come back when, which warps run on which SM. To make a kernel fast, we need a working theory of what the hardware will do with our code.

The frustrating-but-honest truth: there are many ways a kernel can be "slow," and you can't tell which one applies just by looking at the code. You have to *measure*, then *diagnose*. The mental model is what lets you connect a measured number (e.g. "0.713 ms / 189 GB/s") to a specific bottleneck (e.g. "we're not bandwidth-limited; the per-iter dependency chain is").

By the end of Part 2 you should be able to:

1. Read a kernel and predict roughly how many warps will run per SM.
2. Look at a per-iter loop body and identify what the longest serial dependency chain through it is.
3. Hear "this kernel hits 189 GB/s of 1008 peak" and know whether that's good or bad given the workload.
4. Recognise the three diagnostic categories — bandwidth-bound, compute-bound, dependency-chain-bound — and know which optimizations target which.

These are the skills behind every commit in Phase 1, Phase 2, and Phase 3, even when we didn't say so explicitly.

> **Checkpoint 2.1** Before reading further: when a kernel achieves "20% of peak HBM bandwidth," does that mean it's fast, or slow? What would you need to know to decide?

2.2 The hardware: SMs, warps, threads

The RTX 4090 has **128 Streaming Multiprocessors (SMs)**. Each SM is a mostly-independent compute unit with its own register file, shared memory, L1 cache, and four "warp schedulers." All 128 SMs share the L2 cache and HBM.

```
GPU
 ├── 128 SMs
```

```
 │    ├── 4 warp schedulers           (issue 1 warp's instruction
per cycle each)
 │    ├── 65536 32-bit registers      (split among resident
threads)
 │    ├── 100 KB shared memory + L1   (split between the two,
configurable)
 │    └── compute units               (CUDA cores for fp32/int,
Tensor Cores for MMA)
 │
 ├── L2 cache                         (72 MB total, shared across
all SMs)
 └── HBM (main DRAM)                  (24 GB, ~1008 GB/s peak
bandwidth)
```

When you launch a kernel like:

```
dim3 grid(batch, n_heads);   // 8 × 32 = 256 blocks
dim3 block(32);              // 32 threads per block
my_kernel<<<grid, block>>>(...);
```

…the GPU schedules the 256 blocks onto the 128 SMs. Each block stays on the SM it's assigned to until it finishes (no migration). Multiple blocks may run on the same SM at once if they fit (more on this in §2.5).

Inside a block, threads are organised into **warps of 32 threads**. A block of 32 threads = 1 warp. A block of 128 threads = 4 warps. A block of 1024 threads = 32 warps. Warps are the fundamental unit of execution: the SM doesn't issue instructions per-thread, it issues *per-warp* (one instruction at a time across all 32 threads).

Why 32?

Hardware history. The original GPUs had SIMD-32 execution units, and the abstraction stuck. Modern NVIDIA GPUs all have warp size 32. AMD GPUs have warp size 64 (they call them "wavefronts"). The number matters for our purposes because:

- Warp shuffles operate on 32 lanes (`__shfl_xor_sync`, etc.).
- A "warp-wide" memory load that's coalesced reads up to 32 elements per instruction.
- Block sizes are usually multiples of 32 so no warp has idle lanes.

The single most important fact about a warp: **all 32 lanes execute the same instruction in the same cycle**. That's *SIMT* — Single Instruction, Multiple Threads. We'll get back to what happens when the lanes diverge in §2.4.

Mapping the kernels we wrote

Here's how the actual Phase 1 attention kernel `v3` maps to this hierarchy:

```
Launch: grid(batch=8, n_heads=32) = 256 blocks
        block(32) = 32 threads = 1 warp per block

Each block:
  - Runs on one SM. With 256 blocks and 128 SMs, ~2 blocks per SM.
  - Holds 1 warp (32 threads) — single-warp block.
  - Owns one (batch, head) pair: reads q [head_dim], K and V
    [seqlen_kv, head_dim], writes out [head_dim].

Each thread (= one lane within the warp):
  - Owns 4 d-lanes (since head_dim=128, 128/32 = 4).
  - Iterates the seqlen_kv loop, accumulating one output.
```

For Phase 3 v3 (Phase 3c decode GEMM), the block has 128 threads (4 warps) and they share the K-reduction work — different kernel structure, same building blocks.

> **Checkpoint 2.2** - For a kernel launched with `grid(8, 32)`, `block(32)`, how many warps total are dispatched to the GPU? - On the RTX 4090's 128 SMs, what's the *minimum* number of SMs that would have to be active to host that workload? (Hint: at least one warp per used SM, but multiple blocks can share.)

2.3 The memory hierarchy

If you internalise one thing from Part 2, make it this: **memory is a hierarchy with latencies that span 3–4 orders of magnitude**. Where your data lives determines whether your kernel runs in microseconds or milliseconds.

The hierarchy, fastest to slowest, on RTX 4090:

Storage	Scope	Size	Read latency (rough)	Bandwidth
Registers	Per-thread	255 × 32-bit	0 cycles (free)	n/a
Shared mem	Per-block	100 KB / SM (configurable)	~20 cycles	~10 TB/s/ SM
L1 cache	Per-SM	shares 100 KB with shmem	~28 cycles	~10 TB/s/ SM
L2 cache	Per-GPU	72 MB	~190 cycles	~5 TB/s total
HBM (DRAM)	Per-GPU	24 GB	~500 cycles	1008 GB/s peak

The actual cycles vary by access pattern, contention, cache state, and fp16 vs int32 vs … — these numbers are ballpark. But the *ratios* matter: HBM is ~25× slower latency than shmem, and ~100× slower than registers.

A working analogy: **think of HBM as a slow truck and shmem as a fast conveyor belt right next to your workbench**. You don't move things off the workbench unless you have to. You load big batches from the truck once and then work from the conveyor belt.

What "bandwidth-bound" actually means

When we say a kernel is "bandwidth-bound on HBM," we mean: the runtime is set by *how fast we can pull data through the HBM-to-SM pipe*, and no amount of compute optimization will help unless we cut HBM bytes. The 4090's HBM peak is 1008 GB/s. If our kernel needs to move 128 MB through HBM and that takes 0.127 ms, we're at HBM peak. If it takes 0.7 ms, we're at 18% of HBM peak — meaning *something else is the ceiling* and we have headroom.

The cheat-sheet question: **"if we doubled the HBM bandwidth, would the kernel run twice as fast?"** If yes, bandwidth-bound. If no, something else.

Where each thing lives in our kernels

For the Phase 1 v3 decode attention kernel:

```
q   (one [head_dim] = 256 bytes per (batch, head))   →  loaded
into per-thread REGISTERS once at start
K   (whole [seqlen_kv, head_dim] = 1 MB per block)   →  streamed
from HBM via L1/L2
V   (whole [seqlen_kv, head_dim] = 1 MB per block)   →  streamed
from HBM via L1/L2
o_acc, m, l (per-thread running state)               →  REGISTERS
for the whole loop
out (one [head_dim])                                 →  written to
HBM at end (one vec store)
```

Q lives in registers because it's reused over every j iteration. KV is read once and not reused, so streaming from HBM is fine — we don't benefit from caching it. The running softmax state stays in registers because per-iter updates are critical-path.

For the Phase 2b INT8 KV attention, the structure is the same but K and V live as int8 in HBM (half the bytes vs fp16). For Phase 3c W4A16, the activations are *cached in shared memory* because the same K values are reused N times across the output columns the block computes — that reuse is what makes shmem caching worth it.

> **Checkpoint 2.3** - In the v3 decode attention kernel, why is Q held in registers rather than streamed from HBM? - In the Phase 3c W4A16 GEMM kernel, why is `act` worth caching in shared memory but `weight` not? - The 4090's HBM peak is 1008 GB/s. If a decode kernel reads 128 MB of KV per call, what's the *theoretical fastest* it could run?

2.4 SIMT execution: same instruction, 32 lanes

The single-instruction-multiple-thread model is the most important performance abstraction in CUDA, and it has *consequences*.

What "lockstep" means

When a warp executes an instruction like `c = a + b`, all 32 lanes execute that instruction in the same cycle. Each lane has its own copy of `a`, `b`, `c` in its registers — the instruction operates on 32 register copies simultaneously.

So if the warp does:

```
float a = some_value;        // lane 0 has its own, lane 1 has its
own, ...
float b = other_value;
float c = a + b;             // all 32 lanes add, in one cycle
```

The work is "free" in the sense that lane 1 doing this work doesn't make lane 0's work slower. The warp's compute throughput is *all 32 lanes at once*.

This is why we keep saying "**redundant scalar work is essentially free under SIMT**." If lane 0 needs to compute `m_new = max(m, s_j)`, and we have 32 lanes already executing the same warp instruction, then having all 32 lanes redundantly compute the same `m_new` is free. Lanes 1–31 weren't going to do anything more useful on that cycle anyway. (This was the Phase 1 v1 "wrong hypothesis" lesson: trying to *save* this "redundant" work by concentrating it in one lane introduced serialization where there was none.)

Divergence: when lanes don't agree

What if the lanes need to do *different* things? Consider:

```
if (threadIdx.x < 16) {
    a = x + y;
} else {
    a = x - y;
}
```

In the warp, the first 16 lanes want `x + y`; the other 16 want `x - y`. The hardware can't execute two different instructions in one cycle. Instead, it executes the first branch with only lanes 0–15 *active* (lanes 16–31 are masked off — their results aren't written), then the second branch with lanes 16–31 active. The cost: **2× the cycles** for that block of code.

Divergence is most painful when it's complex or unpredictable. Predictable divergence (e.g. one branch is rare) costs less because the "both branches" cost is amortized.

For our kernels, we mostly *avoid* divergence by design: - Bounds checks at the top of the kernel (`if (n >= N) return;`) diverge but only at the edges, so most blocks aren't affected. - The per-iter loop body has no `if`s that depend on per-thread state — every lane does the same dot-product, the same softmax update, the same FMA, just on its own data.

Warp shuffles: cross-lane communication

Within a warp, lanes can exchange register values without going through shared memory using **warp shuffles**:

```
float partial = q_val * k_val;
partial = __shfl_xor_sync(0xFFFFFFFF, partial, 16);   // swap with lane^16
partial = __shfl_xor_sync(0xFFFFFFFF, partial, 8);    // swap with lane^8
... // continue with offsets 4, 2, 1
// Now all 32 lanes have the same value: the sum across the whole warp.
```

A shuffle is a "butterfly" pattern: with 5 shuffle ops (offsets 16, 8, 4, 2, 1), every lane ends up with the sum (or max, etc.) across the whole warp. This is *much* cheaper than going through shared memory because there's no shmem store + sync + load — it's all register-to-register over the warp's internal datapath. **5 cycles vs ~30+ for shmem reduction.**

Our kernels use this in `warp_reduce_sum` and `warp_reduce_max`. The v3 attention kernel's entire dot-product reduction is one `warp_reduce_sum` call — no shmem involved.

> **Checkpoint 2.4** - When 16 lanes take a branch and 16 don't, how many cycles does the branch cost compared to all 32 taking the same path? - `warp_reduce_sum` does 5 shuffle operations. Why exactly 5? (Hint: what powers of 2 are involved?) - In Phase 1 v3, we said "redundant scalar work across the warp is free." Give a concrete example from a kernel where this principle matters.

2.5 Occupancy and latency hiding

Threads and warps don't run continuously — they spend a lot of time *stalled*, waiting for memory loads to complete, waiting for the result of an `__expf`, etc. The GPU's main strategy for hiding these stalls is to keep many warps resident on an SM and switch between them when one stalls.

What “resident” means

An SM has a finite budget — registers, shared memory, warp slots. When a block is launched onto an SM, it *uses* some of that budget:

```
Resources used by one block of K threads, R registers/thread, S
bytes shmem:
  registers    : K · R                  (out of 65536)
  shared mem   : S                      (out of 100 KB)
  warp slots   : ceil(K / 32)           (out of 48 warps/SM on
Ada)
  block slots  : 1                      (out of 16-24 blocks/SM on
Ada)
```

The SM holds as many blocks as fit under all four constraints. The **occupancy** is the resulting `active warps / max warps`:

```
occupancy = (blocks_per_SM · warps_per_block) / 48
```

For v3 (single-warp block, 33 registers/thread): the constraint is the block-slot limit of 16 (or whatever the Ada hard cap is on small blocks). 16 blocks × 1 warp = 16 warps. Occupancy = 16/48 ≈ 33%.

For Phase 1 v0 (128-thread block, 29 registers/thread): up to 12 blocks × 4 warps = 48 warps = 100% occupancy.

How occupancy helps

When a warp issues a memory load that misses in cache, it takes hundreds of cycles to come back. During those cycles, the warp scheduler looks at the *other* resident warps and picks one whose next instruction is ready to issue. If there’s no ready warp, the SM stalls.

With many resident warps, there’s almost always *some* warp ready, so the SM keeps issuing instructions. **High occupancy** = **latency hiding via parallel waiting**.

But there’s a saturation: once you have enough warps that the SM is never stalled waiting, more warps don’t help. Past that point, occupancy is just overhead (register pressure, more shmem competition, etc.).

When occupancy matters and when it doesn't

It matters when the kernel has lots of memory-load latency to hide. If every loop iteration has a cache-missing load with a 500-cycle latency, you want enough warps that the SM can do ~16 instructions per cycle across them while one waits.

It *doesn't matter* when the per-iter dependency chain is the bottleneck. If each warp's next instruction depends on the previous one (true serial), no amount of additional warps helps — the SM still issues 4 instructions per cycle, but one warp's instructions can only proceed in sequence.

This was the Phase 1 v3 lesson: we traded 4 warps/block (full occupancy) for 1 warp/block (33% occupancy) and *gained* 1.50× speedup. The lost latency hiding didn't matter because the per-iter chain was the ceiling. **Occupancy is a means, not an end.**

How to read the bench output

When we say v3 is at "33% occupancy" we mean: of the 48 warp slots each SM could hold, only 16 are active. The remaining slots are wasted, but that's not the same as a slow kernel — it's only slow if those extra warps would have helped hide latency, which they don't here.

> **Checkpoint 2.5** - A kernel uses 60 registers per thread. With 65536 registers per SM, how many threads can it have resident? (Ignore other constraints.) - Phase 1 v3 has 33% occupancy and is faster than v2 at 100% occupancy. Explain this in one sentence. - When would *adding* warps make the kernel slower?

2.6 Synchronization: what __syncthreads costs and means

When threads in different warps need to coordinate — write to shmem in one warp, read in another — you need a barrier. The two main primitives:

`__syncthreads()`

Block-wide barrier. **Every thread in the block must reach the `__syncthreads` before any thread proceeds past it.** Internally:

- The SM tracks how many warps have hit the barrier.
- Warps that arrive early are parked (the scheduler picks other warps).
- When all warps in the block have hit, all are released.

Cost: typically tens of cycles (it depends on how synchronised the warps were going in). For our kernels, the bigger cost is usually the *consequence* — the compiler can't reorder loads or stores across a `__syncthreads()`. That's a load barrier, not just a sync barrier.

This is the Phase 1 v1 lesson: V loads inside the per-`j` `__syncthreads()` couldn't be hoisted by nvcc above the sync, so V latency couldn't hide behind the K reduction. The fix was to manually move the V load to the top of the iteration — same code, just outside the barrier.

`__syncwarp()`

Warp-wide barrier (cheaper). For single-warp blocks, mostly redundant because the lanes are already in lockstep. But there are subtle cases (e.g., after writing to shmem from one lane, before another lane reads it) where you need an explicit warp sync to defeat compiler reordering.

Warp shuffles as cross-lane sync

The `__shfl_xor_sync` calls we discussed in §2.4 don't need a separate sync — the "sync" is implicit in the shuffle's mask (the first arg, typically `0xFFFFFFFF` to require all 32 lanes participate). This is the cheapest way to communicate across a warp.

A subtlety: `__syncthreads()` and divergence

`__syncthreads()` deadlocks if some threads in the block can't reach it (because they returned early, for example). For this reason, you'll see patterns like:

```
// BAD — deadlocks if some threads return early
if (n >= N) return;
```

```
... do work ...
__syncthreads();

// OK — all threads reach the sync
const bool valid = (n < N);
... do work guarded by valid ...
__syncthreads();
if (valid) { ... }
```

We hit this in the Phase 3c kernel where some threads might be out of the N range. The fix was to guard with `valid` and have all threads participate in the sync.

> **Checkpoint 2.6** - You write a kernel where every thread writes to its own shmem slot, then every thread reads that same slot back. Do you need `__syncthreads()`? - In Phase 1 v1, why does putting the V load *before* the `__syncthreads()` matter for performance? - What goes wrong if `__syncthreads()` appears inside an `if` branch that only some threads take?

2.7 The dependency chain

This is the concept that *most* of our optimizations turn on, and the one that's least taught in a 101 course.

What "dependency chain" means

In a per-iteration loop body, there's often a sequence of operations where each depends on the previous:

```
load → unpack → multiply by scale → FMA → softmax update → next iter
```

Each of those operations has some *latency* — the number of cycles from issuing the instruction to having its result available. The *total* latency through the chain is the sum of the latencies.

If the chain is short, the next iter starts soon and throughput is high. If the chain is long, the next iter waits.

The “instruction issue rate” view

The SM can issue ~4 warp instructions per cycle (one per warp scheduler). If the warps have lots of *independent* work, the SM is saturated — every cycle, 4 ops are issued. That’s “compute-bound.”

But if the warps’ work is *serially dependent* (instruction N depends on instruction N-1’s result), then each warp can only have one instruction in flight at a time, and the warp issues at the rate of the chain (1 / chain_length per warp-cycle).

The SM can compensate by switching between many warps — each warp has its own chain, and the SM rotates through them. With enough resident warps, the SM stays saturated even if each individual warp’s chain is long. *This is what occupancy buys you.*

When does that not work?

- If each warp has only a tiny number of independent ops, you need lots of warps to fill the SM. The kernel becomes occupancy-limited.
- If memory loads are part of the chain, those add hundreds of cycles of latency — even more warps needed.

“Dependency-chain-bound” as a category

This is the third category beyond bandwidth-bound and compute-bound: the kernel is bottlenecked by **how fast the per-iter chain can complete**, regardless of how much compute or bandwidth is available.

How to recognise it: - HBM is not at peak (so not bandwidth-bound). - Compute units are not at peak (so not compute-bound). - The kernel doesn’t speed up with more SMs (Phase 1 v4) or more parallel loads (Phase 1 v5). - The kernel *does* speed up when you make the chain shorter (Phase 1 v3’s single-sync reduce, Phase 2c’s per-group K scale).

Phase 1 v3 → v2: the gap was a `__syncthreads()` in the middle of the chain. Removing it cut the chain. **The 570 µs we measured wasn’t “sync overhead” — it was “what the chain was waiting for.”**

Phase 2c: by moving K scales out of the per-iter loop (loading them once per group instead of every iter), we cut a load+multiply from the chain. The structural shorter-chain made the kernel faster, not the fewer bytes loaded.

The diagnostic question

For any kernel, ask: **what's the per-iter critical path?** Then ask: **what would shorten it?**

For decode attention: the per-iter chain is `load K` → `dot product + warp_reduce_sum` → `softmax recurrence` → `FMA with V` → `next iter`. Each step is a few cycles or more (warp_reduce_sum is ~5 cycles for the shuffle tree). Total per-iter chain: ~30-50 cycles, depending on loads and dependencies.

For W4A16 GEMM: the per-iter chain is much shorter (no softmax, no reduction within a thread — just FMA). That's why W4A16 wins big on memory-bound shapes: it has no chain ceiling and the byte savings translate directly.

> **Checkpoint 2.7** - What's the per-iter dependency chain in the v3 attention kernel? - Why did the "single-sync reduce" in v2 produce *more* speedup than just removing a `__syncthreads()` would predict? - Phase 2b (INT8 KV) halved KV bytes but didn't speed up the kernel. Where was the chain hiding the win?

2.8 The diagnostic framework

You now have the vocabulary to talk about where a kernel's time goes. The framework:

Three bottleneck categories

1. **Bandwidth-bound**: the kernel is reading data through HBM at close to peak (or close to L2 peak, or whatever the relevant memory level is). Cutting bytes helps. Adding more SMs doesn't. *Diagnostic*: achieved bandwidth ≈ peak.

2. **Compute-bound**: the kernel is doing arithmetic close to the GPU's FLOPS peak. Adding more bandwidth doesn't help. Tensor Cores (if applicable), wider vectorization, or fewer ops do. *Diagnostic*: achieved TFLOPS ≈ peak compute.

3. **Dependency-chain-bound**: neither of the above. The per-iter serial chain limits throughput. Shorter chains (fewer ops per iter, moving work out of the loop) help. More SMs / more bandwidth don't. *Diagnostic*: well below both HBM and compute peaks, kernel doesn't speed up with bigger grid or wider loads.

For decode attention on the 4090: - v3 at 189 GB/s of 1008 GB/s = 19% of HBM peak. NOT bandwidth-bound. - Compute-wise, attention's per-iter ops are tiny. NOT compute-bound. - Therefore dependency-chain-bound. The optimization lever was *the chain*: v3 cut the cross-warp shmem broadcast (v2 → v3), v3 used vectorized loads + single warp to keep the chain compact, v4/v5 failed because they didn't shorten the chain.

For W4A16 GEMM on the 4090 at M=1: - 1577 GB/s of 1008 GB/s = 156% of HBM peak (L2-served). Effectively bandwidth-bound on L2. - The kernel's win comes from cutting weight bytes 4× (fp16 → INT4). - Compute is trivial.

How to apply the framework

When you measure a new kernel and it's slower than you hoped:

1. **Calculate achieved bandwidth** from bytes moved / time. Compare to peak. If close to peak: bandwidth-bound. Cutting bytes is your tool.
2. **Calculate achieved compute** (FLOPS / time). Compare to peak. If close: compute-bound.
3. **If neither**: dependency-chain-bound. Look at the per-iter loop body and find what depends on what. The chain is your enemy.

When picking an optimization: - Don't add bandwidth-targeting work to a kernel that isn't bandwidth- bound. (Phase 1 v4 split-K, Phase 1 v5 cp.async — both lost.) - Don't add compute-targeting work to a kernel that isn't compute- bound. (Tensor Cores at decode M=1 — wrong tool.) - *Do* attack the chain — but recognise what's in the chain (loads, shmem hops, reductions, `__expf`, etc.).

2.9 Putting it together: reading a kernel like a performance engineer

A working checklist for sizing up any GPU kernel:

1. Block geometry - `grid` size, `block` size, total warps? - Reasonable for the GPU? (e.g. 256 blocks of 4 warps each on 128 SMs is fine; 32 blocks of 1 warp each is severely under-occupied.)

2. Per-thread work - What does each thread own? Registers used? - Does each thread loop?

3. Memory pattern - What's read from HBM, how many times? - What's cached in shmem, L1, registers? - Coalesced loads across the warp?

4. Per-iter critical path - What's the longest serial dependency through one loop iter? - How many cycles does it take?

5. Synchronization - `__syncthreads()` calls — how many per iter? Why? - Any places where loads sit *inside* a sync that could be hoisted out?

6. Diagnostic guess - Bandwidth, compute, or chain bound? - What evidence (from the code) supports that guess? - What's the right optimization given the guess?

Try applying this checklist to your favorite of the v0–v5 attention kernels before reading on. (Hint: they all have different answers for items 1, 4, and 5.)

2.10 Summary of Part 2

By the end of Part 2 you should be comfortable with:

- The hierarchy: **GPU → SMs → blocks → warps → threads**, and how a kernel launch maps to this.
- **SIMT execution**: 32 lanes in lockstep, divergence costs, why redundant warp-wide work is free.
- **Memory hierarchy**: registers / shmem / L1 / L2 / HBM, with latencies spanning 3–4 orders of magnitude. Where to put what.
- **Occupancy and latency hiding**: more resident warps → more in-flight work → better at hiding memory latency. Capped by register/shmem/warp budgets.
- **Synchronization**: `__syncthreads()` is a barrier *and* a load barrier. Warp shuffles are cheaper than shmem reductions.
- **The dependency chain**: per-iter serial path that limits throughput when neither bandwidth nor compute is at peak.

- **The diagnostic framework**: bandwidth-bound vs compute-bound vs dependency-chain-bound — and how to tell which applies.

Part 3 will put this to work: we'll walk through the v0 → v5 attention journey, applying these tools at each step. By the end you should be able to predict *why* an optimization will or won't help, before measuring.

2.11 Glossary additions

Term	Meaning
SM	Streaming Multiprocessor. The 4090 has 128.
Warp	32 threads executing in SIMT lockstep.
Lane	One thread within a warp; `threadIdx.x % 32`.
Block	Set of warps that share an SM and shared memory.
Grid	Set of blocks. The kernel launch dispatches the grid.
Resident warp	A warp the SM is currently holding (vs queued or finished).
Occupancy	`active warps / max warps per SM`.
HBM	High Bandwidth Memory. The GPU's main DRAM.
Coalesced load	A warp-wide load where the 32 lanes' addresses are contiguous, served as one wide transaction.
Bandwidth-bound	Kernel runtime is set by data-movement throughput.
Compute-bound	Kernel runtime is set by arithmetic throughput.
Dependency-chain-bound	Kernel runtime is set by the per-iter serial dependency path.
SIMT	Single Instruction, Multiple Threads. NVIDIA's execution model.
Divergence	Lanes within a warp taking different branches; serializes the branches.

Term	Meaning
Warp shuffle	Register-to-register communication across lanes within a warp.
`__syncthreads()`	Block-wide barrier. Also prevents the compiler from reordering loads across it.
Latency hiding	Using other warps' work to fill in time while one warp waits on memory.

Ready for Part 3? Part 3 walks the v0 → v5 attention kernel evolution using Part 2's framework. Each kernel gets a paragraph or two of "here's what the bottleneck was at this step, here's the lever we pulled, here's why it worked (or didn't)."

Before continuing, work through the checkpoint questions in §2.5, §2.7, and §2.8 without looking — those three are the load-bearing ones for Part 3.

Part 3 — Decode attention, naive → fast

This is where Parts 1 and 2 come together. We walk through every kernel version we wrote — v0 through v5 — and at each step we'll do four things:

1. **State the bottleneck of the previous version** using the diagnostic framework from §2.8 (bandwidth / compute / dependency- chain).
2. **Form the hypothesis** for the change — what we believe should move the bottleneck.
3. **Sketch the implementation** — enough to re-derive the kernel from memory.
4. **Reconcile** — was the hypothesis right? Read the numbers; learn the lesson.

After Part 3 you should be able to:

- Walk through the v0 → v3 progression from memory.
- Recognise the v1 / v4 / v5 wrong-hypothesis traps when they appear in new contexts.

- Explain to a colleague why v3 wins where v4 doesn't.

> **Heads up**: this part is the longest in the book. Don't try to read it in one sitting. v0–v3 form one arc (the wins); v4–v5 form a shorter second arc (the regressions). Take a break between them.

3.1 v0 — the naive two-pass softmax baseline

What we're building

The contract is fixed: for one (batch, head) pair, compute

```
s[j]   = scale · dot(q, K[j, :])              for j in
0..seqlen_kv-1
m      = max(s)
p[j]   = exp(s[j] − m) / Σ exp(s[j] − m)
o[d]   = Σ_j p[j] · V[j, d]                   for d in
0..head_dim-1
```

`q` has shape `[head_dim]`, K and V have shape `[seqlen_kv, head_dim]`, `out` has shape `[head_dim]`. Llama 3 8B head config: `head_dim = 128`, plus the GQA index map `kv(h) = h / 4` to pick the right KV head.

Block geometry

The natural mapping: - **Grid**: `(batch, n_heads)`. One block per (batch, head) pair. For `batch=8, n_heads=32` we get 256 blocks. - **Block**: `(head_dim) = 128 threads = 4 warps`. Each thread owns one output lane `d = threadIdx.x`.

This gives every thread a clear role: thread `d` is responsible for producing `o[d]`, the d-th output element.

Three phases (the naive structure)

The v0 algorithm has three sequential phases, with shared memory holding the score vector `s[0..seqlen_kv)` between them:

Phase 1 — compute scores. For each `j` in 0..seqlen_kv-1: - Every thread computes a partial dot product `q[d] * K[j, d]`. - The 128 threads reduce across the block (warp reduce + shmem combine) to get the full `dot(q, K[j, :])` as a single scalar. - Multiplied by `scale = 1/√head_dim` and stored in `s_smem[j]`.

This loop costs **two `__syncthreads()` per j** — one to gather warp partials into shmem, one to broadcast the final result.

Phase 2 — softmax. Now `s_smem[0..seqlen_kv)` is fully populated. - Block-reduce max over `s_smem` → `m`. - Each thread computes `s_smem[j] := exp(s_smem[j] − m)` for its strided j (and writes back). Block-reduce sum over the exp'd values → `l`. - We'll divide by `l` in Phase 3.

Phase 3 — output. For each `j` in 0..seqlen_kv-1: - Every thread loads `p[j] = s_smem[j] / l` (already in shmem). - Each thread reads `V[j, d]` (where `d = threadIdx.x`). - Accumulates `o += p[j] · V[j, d]` into a per-thread fp32 register.

After the loop, each thread writes its `o[d]` to global memory as fp16.

Memory inventory

- `q`: loaded into per-thread registers at kernel start (each thread holds `q[threadIdx.x]`, one scalar per thread).
- `K`: streamed from HBM through L2/L1 in Phase 1. Each thread reads `K[j, threadIdx.x]` per j-iter.
- `V`: streamed from HBM through L2/L1 in Phase 3. Each thread reads `V[j, threadIdx.x]` per j-iter.
- `s_smem`: dynamic shared memory of size `seqlen_kv × 4 bytes`. For `seqlen_kv = 4096`, that's 16 KiB. Below the 48 KiB default cap.
- `reduce_smem`: a small scratch for the block reduction (one slot per warp, so 4 floats = 16 bytes).

What kind of kernel is this?

Using §2.8: is v0 bandwidth-bound, compute-bound, or chain-bound?

The Phase 1 loop reads `K` once and Phase 3 reads `V` once — both at fp16. Total HBM traffic per (batch, head) ≈ `2 × seqlen_kv × head_dim × 2 bytes`. For `seqlen_kv = 4096, head_dim = 128`: 2 MiB per (batch, head), ~256 MiB across the 256 blocks.

If the kernel were bandwidth-bound and pulled at HBM peak (1008 GB/s), this would take 256 MiB / 1008 GB/s = ~0.25 ms.

Measured: **1.669 ms**. So we're at ~256 MiB / 1.669 ms ≈ 153 GB/s effective bandwidth, or 15% of HBM peak.

Not bandwidth-bound. Not compute-bound either (the per-iter ops are trivial — 128 FMAs per `j`, much less than the SM's compute capacity).

So: **dependency-chain-bound**. The per-`j` chain is `K load → block reduce (with 2 syncs) → shmem write`. Two `__syncthreads()` per j × 4096 j's = 8192 sync barriers per block. *That's* where the time goes.

Result

1.669 ms / 80 GB/s achieved KV bandwidth at the reference workload. Max |abs diff| vs the fp32 reference is 6e-5 (well within the `rtol/atol = 2e-2` correctness gate). 2.26× faster than PyTorch eager (3.77 ms); 23% behind PyTorch SDPA (1.36 ms).

This is our **CUDA-vs-CUDA baseline**. Everything from here on is relative to v0.

> **Checkpoint 3.1** - In v0, why do we need shared memory for the score vector? What would happen if we just kept the partial scores in registers per-thread? - What's the per-`j` dependency chain in v0? How many cycles is it roughly (count the syncs)? - The kernel hits 15% of HBM peak. Is v0 bandwidth-bound? Use §2.8's framework to argue.

3.2 The online softmax trick (Milakov–Gimelshein 2018)

Before we modify v0, we need to learn the math that powers v1 and everything after. This is the *single most important algorithmic trick* in modern attention kernels — every FlashAttention paper builds on it.

The problem with v0

v0 has three sequential passes over the score vector: 1. Phase 1 builds `s_smem`. 2. Phase 2 reads `s_smem` to find max + sum exp. 3. Phase 3 reads `s_smem` again (now as `p`) to compute output.

That's *three passes over an* `seqlen_kv`*-sized array in shmem*. It also means `seqlen_kv` is capped by shared memory — at the 48 KiB default we can't go past ~12k positions. For long context (32k+ tokens), this doesn't scale.

We want **one pass**. But softmax is a non-local operation: the denominator `Σ exp(s[j] − max)` depends on `max(s)`, which needs to see all of `s` first. Or does it?

The recurrence

Suppose we're processing `s[j]` one at a time and we have, so far: - `m` = max of `s[0..j-1]` - `l` = `Σ_{j' < j} exp(s[j'] − m)` - `o_acc[d]` = `Σ_{j' < j} exp(s[j'] − m) · V[j', d]`

Now a new score `s_j` comes in. Two cases:

Case 1: `s_j ≤ m`. Then `m` stays the same. We just update:

```
l         += exp(s_j − m)
o_acc[d]  += exp(s_j − m) · V[j, d]
```

Case 2: `s_j > m`. The max changes. Let `m_new = s_j` (or in general, `max(m, s_j)`). Our existing `l` and `o_acc` were computed relative to the *old* `m`, so they're out of date. We rescale them:

```
m_new     = max(m, s_j)
α         = exp(m − m_new)           // rescale factor for old
contributions
l_new     = l · α + exp(s_j − m_new)
o_acc[d]  = o_acc[d] · α + exp(s_j − m_new) · V[j, d]
m         = m_new
```

Case 1 is just case 2 with `m_new = m`, so `α = exp(0) = 1` and the rescale is a no-op. We can use the case-2 formula always:

```
For each j in 0..seqlen_kv:
    m_new = max(m, s_j)
    α     = exp(m - m_new)
    l     = l · α + exp(s_j - m_new)
    o_acc[d] = o_acc[d] · α + exp(s_j - m_new) · V[j, d]
    m     = m_new

At end: o[d] = o_acc[d] / l
```

That's the **online softmax recurrence**. Single pass over `j`, no shmem score buffer needed. The state per (batch, head) is just three scalars (`m`, `l`, plus per-thread `o_acc[d]`) in registers.

Why this works numerically

The exponent argument `s_j - m_new` is always `≤ 0` (since `m_new ≥ s_j`), so `exp(...) ≤ 1`. No overflow. The rescale `α = exp(m - m_new) ≤ 1` only shrinks old contributions, never blows them up.

Compare to the "naive" softmax `exp(s_j) / Σ exp(s_j)`: with large `s_j` (which happens in attention because of the dot product) you'd overflow `exp` instantly. The max-subtraction is what makes softmax numerically stable in practice.

Initialisation

At `j = 0`: - Start `m = -∞`, `l = 0`, `o_acc = 0`. - First iteration: `m_new = max(-∞, s_0) = s_0`. `α = exp(-∞ - s_0) = 0`. `l = 0 · 0 + exp(0) = 1`. `o_acc = 0 · 0 + 1 · V[0, :] = V[0, :]`.

Self-consistent. The first `j` becomes the running state.

> **Checkpoint 3.2 (load-bearing!)** Without looking, write the online softmax recurrence on paper. Five assignments: `m_new`, `α`, `l`, `o_acc[d]`, `m`. You'll re-derive this many times in the rest of Part 3.

3.3 v1 (naive port) — the regression

Hypothesis

We have the math. Let's just port it into v0's structure.

Predicted outcome: One pass over KV (no Phase 2 separate softmax, no Phase 3 separate output reduction), no shmem score buffer needed. *Should* be faster than v0 — strictly less work.

Implementation sketch

Block geometry: same as v0 (`grid(batch, n_heads), block(head_dim)`). Each thread still owns one output lane `d`.

The body:

```
m_state[d] = -inf
l_state[d] = 0
o_state[d] = 0       // per-thread, one fp32 register

for j in 0..seqlen_kv:
    // Block-wide dot product: same as v0 Phase 1's per-j step
    partial = q[d] · K[j, d]
    s_j     = scale · warp_reduce_sum(partial) ... block
reduce ...
    __syncthreads()               // sync to broadcast s_j to all
threads
    s_j     = s_bcast             // every thread reads
broadcasted s_j

    // Online softmax recurrence (every thread runs it in
lockstep)
    m_new  = max(m_state, s_j)
    α      = exp(m_state - m_new)
    p_j    = exp(s_j - m_new)
    o_state = o_state · α + p_j · V[j, d]
    l_state = l_state · α + p_j
    m_state = m_new

out[d] = (half) (o_state / l_state)
```

Result

2.078 ms / 65 GB/s — 0.80× of v0. Regression. Same hardware, same math, supposedly less work, slower.

Diagnosing the regression — the wrong hypothesis

First hypothesis (mine when this happened): “every thread runs the same softmax recurrence redundantly. With 128 threads doing the same scalar work, we’re wasting compute. Let’s move the recurrence to one thread and broadcast.”

Tested. Moved `(m, l, α, p_j)` to lane 0 of warp 0, with shmem broadcast.

Result of the fix: **2.26 ms — worse**.

Why? §2.4 explains it: SIMT-parallel ALU is free. The 128 threads weren't "wasting compute" — they were going to sit at the `__syncthreads()` barrier anyway. Doing identical scalar arithmetic alongside cost nothing. *Moving* the work to one lane introduced serialization where there was none (only lane 0 active during the recurrence, 31 other lanes in the warp idle), AND added shmem hops for the broadcast.

Lesson 1 (load-bearing): when you remove "redundant" work, ask: *were the redundant threads doing nothing useful otherwise?* If yes, the work was free, and "removing" it makes things worse.

Diagnosing the regression — the right hypothesis

Look more carefully at v1's per-`j` chain:

```
1. Load K[j, d]                                          HBM/L2 read
2. Compute partial = q[d] · K[j, d]                        FMA
3. warp_reduce_sum + cross-warp shmem reduce              ~10 cycles +
sync
4. __syncthreads()                                         sync (load
barrier!)
5. Load broadcast s_j                                      shmem read
6. exp(m_state - m_new), exp(s_j - m_new)                  2× expf, ~10
cycles each
7. Load V[j, d]                                            HBM/L2
read          ← latency hidden by ???
8. FMA o_state += p_j · V[j, d]                            FMA
9. Update m, l, m_state                                    ~3 ops
10. Next iter
```

Step 7 — the V load — is *inside the per-iter dependency chain after the* `__syncthreads()`. Recall §2.6: nvcc treats `__syncthreads()` as a *load barrier* — it cannot hoist a load above it. So V latency (hundreds of cycles for an HBM/L2 miss) sits as a single sequential step in the chain.

In v0, V loads happened in Phase 3, *outside* any per-j sync. Phase 3's loop body was just `o += s_smem[j] · V[j, d]` — no syncs, so the compiler aggressively pipelined V loads. Latency hidden.

In v1, V loads sit inside the sync. **Latency exposed.**

The fix: prefetch V

Issue the V load *at the top of the iteration*, before any sync. The load is non-blocking; the value isn't consumed until later. Latency hides behind everything else in the chain.

```
for j in 0..seqlen_kv:
    v_j_register = V[j, d]            // ← load issued NOW,
asynchronously
    // ... K load, reduce, sync, softmax ...
    o_state = o_state · α + p_j · v_j_register
```

That's it. One line moved.

Result with prefetch (commit `ad9c57f`)

1.637 ms / 82 GB/s — 1.02× over v0. Online softmax now does what theory predicts: roughly tied with the two-pass version, with the structural wins of single-pass and unbounded `seqlen_kv`.

What we learned

Lesson 2 (load-bearing): `__syncthreads()` is also a *load barrier*. nvcc won't hoist memory loads above it. If a load is on the per-iter critical path *after* a sync, it serialises with the chain. The fix is manual: move the load before the sync, into a register, and use it after.

This lesson recurs: - In v5, we'll try `cp.async` (explicit async loads) — but for different reasons than what bit us here. - In the Phase 3 W4A16 GEMM, we won't have the sync barrier problem because we don't need cross-warp sync at all.

> **Checkpoint 3.3** - In v1's per-iter chain, where exactly does the V load sit relative to the `__syncthreads()`? - If the V load latency is ~500 cycles and the rest of the chain is ~50 cycles, what's v1's per-iter cost (in cycles) without the prefetch fix? With it? - The "redundant exp" fix made v1 *slower*. State why in one sentence referencing §2.4.

3.4 v2 — single-sync block reduce

What's left to remove

v1 with prefetch has **two `__syncthreads()` per iter**: - One after each warp writes its partial dot-product to `reduce_smem` (so warp 0 can read them all). - One after warp 0 writes the final `s_j` to a broadcast slot (so all warps can read it).

Two syncs per iter × 4096 iters = 8192 sync barriers per block. Can we get to one?

Two-part hypothesis

Part A — Make every warp redundantly compute the final reduce. Instead of warp 0 doing the final reduction and then broadcasting via shmem, have every warp read all four partial values from `reduce_smem` (4 floats — one per warp) and run its own `warp_reduce_sum`. SIMT means every lane in every warp gets the same answer with no shmem round-trip.

Why is this free? The other warps were going to sit at the second `__syncthreads()` anyway. Having them do four shmem reads and a 5-step shuffle reduction costs nothing wall-clock-wise (SIMT lesson again).

Part B — Double-buffer the `reduce_smem`. Without part A, dropping one sync would create a race: iter `j+1`'s write to `reduce_smem` could clobber iter `j`'s reads in slower warps.

Fix: use two slots in `reduce_smem` (size `[2][4]`). Iter `j` writes slot `j & 1` and reads slot `j & 1`. Iter `j+1` writes the *other* slot. The hazard between iter `j`'s read and iter `j+2`'s write (same slot) is gated by iter `j+1`'s sync — there's always a sync between them.

So: each iter writes its slot, syncs once, reads its slot, computes. **One sync per iter.**

Implementation sketch

```
__shared__ float reduce_smem[2][32];    // 2 slots × WARP_SIZE entries

for j in 0..seqlen_kv:
```

```
    v_j = V[j, d]                                          // prefetch
(v1 lesson)
    partial = q[d] · K[j, d]
    partial = warp_reduce_sum(partial)
    if lane_id == 0:
        reduce_smem[j & 1][warp_id] = partial          // double-
buffered write
    __syncthreads()                                        // one sync
per iter
    // Every warp reads all 4 partials and reduces:
    r = (lane_id < 4) ? reduce_smem[j & 1][lane_id] : 0
    r = warp_reduce_sum(r)
    s_j = r · softmax_scale
    // ... online softmax + V FMA (same as v1) ...
```

What we predicted vs measured

Prediction: one fewer `__syncthreads()` per iter. With each sync costing ~50 cycles (rough estimate), 4096 iters × ~50 cycles ≈ 200k cycles per block ≈ 150 μs saved.

Measured: **1.069 ms** (vs v1-prefetch's 1.637 ms). Saved **570 μs**.

What gives? We predicted ~150 μs of sync savings; we got ~570 μs.

The shmem-hop lesson

The extra savings came from removing the *broadcast* shmem write/read, not just the sync. In v1:

```
s_j = shfl_tree → write to broadcast_smem → SYNC → all threads
read broadcast_smem → use s_j
```

In v2:

```
s_j = shfl_tree → directly into register → use s_j
```

That's one shmem write + one shmem read removed from the *critical path*. Shmem accesses are ~20 cycles each; with 4096 iters, removing two shmem ops per iter ≈ 160k cycles ≈ 60 μs.

Plus the sync removal: 150 μs.

Plus reduced instruction count + better scheduling around the shorter chain. The combined gain is more than the sum of the parts — which is what happens when you shorten a critical path: everything that was bottlenecked on it becomes faster.

What we learned

Lesson 3 (load-bearing): removing a sync from the inner loop often saves more than the sync's own cycles. The chain shortens; the chain's *consequences* (instruction scheduling, in-flight load count limits, warp slot pressure) all loosen. **Look at the chain, not the sync.**

Result

1.069 ms / 126 GB/s — 1.53× over v1-prefetch, 1.56× over v0. Now 1.27× faster than PyTorch SDPA on this workload.

> **Checkpoint 3.4** - In v2's per-iter chain, how many sync barriers are there now? - Why does double-buffering `reduce_smem` allow us to drop a sync? (Hint: race condition between iters.) - The actual speedup (~570 μs saved) was larger than predicted (~150 μs from sync alone). What accounts for the rest?

3.5 v3 — vectorized loads + single-warp blocks

The bet

v2 is at 126 GB/s on a 1008 GB/s HBM peak. Still 13% of peak. The remaining bottleneck must still be the chain — not bandwidth.

Two things might shorten the chain further: 1. **Vectorize the K and V loads** — load 4 fp16 values per LDG.E.64 (one uint2 = 8 bytes) instead of one LDG.E.16 (2 bytes). Fewer load instructions per iter. 2. **Eliminate cross-warp sync entirely** — make the block one warp. Then there's no `__syncthreads()` needed at all (warp internal communication via shuffle is sync-free).

But going to single-warp blocks means: each thread owns 4 d-lanes (`head_dim / 32 = 4`) instead of 1. Per-thread work 4×. Per-block warp count drops from 4 to 1 — **occupancy drops from full (48 warps/SM) to ~33%.**

The occupancy concern

§2.5 says: lower occupancy means less latency hiding. With 33% occupancy, fewer warps are resident to fill in while one stalls on a memory load.

The bet: **the lost latency hiding doesn't matter because the chain ceiling is what's bottlenecking us anyway.** Those extra warps in v2 weren't hiding latency — they were sitting at `__syncthreads()`. Trade their slots for shorter chain.

Implementation sketch

```
constexpr int VEC = 4;                  // 4 fp16 per thread per
load = LDG.E.64

Block: 32 threads (= 1 warp).
Each thread owns lanes [tid*4 .. tid*4+3] (4 d-lanes).

q is loaded once at start as a float4 (4 fp16 from HBM → 4 fp32 in
registers).

for j in 0..seqlen_kv:
    v_j = load_half4_as_float4(&V[j, tid*4])         // ← prefetch
(v1 lesson)
    k_j = load_half4_as_float4(&K[j, tid*4])
    partial = q.x·k_j.x + q.y·k_j.y + q.z·k_j.z + q.w·k_j.w
    partial = warp_reduce_sum(partial)               // single-
warp = no sync!
    s_j = partial · scale
    // online softmax + V FMA on 4 lanes:
    α = exp(m_state − m_new); p_j = exp(s_j − m_new)
    o_state.x = o_state.x · α + p_j · v_j.x
    o_state.y = o_state.y · α + p_j · v_j.y
    o_state.z = o_state.z · α + p_j · v_j.z
    o_state.w = o_state.w · α + p_j · v_j.w
    l_state = l_state · α + p_j
    m_state = m_new
```

```
write 4 output lanes via store_float4_as_half4
```

Note: zero `__syncthreads()`. Zero shared memory. Just registers and warp shuffles.

Result

0.713 ms / 189 GB/s — 1.50× over v2, 2.34× over v0, 1.91× faster than PyTorch SDPA. Max |abs diff| = 3e-5.

The bet paid off. The 33% occupancy was fine because the lost warps were idle at syncs anyway.

Why the win this time?

Three things stacked: 1. **No `__syncthreads()`** — chain shortened by the entire sync barrier (~50 cycles), every iter. 2. **No shmem at all** — no reduce_smem, no broadcast slot. register-to-register all the way. 3. **Wider per-thread loads** — vectorized K/V means fewer load instructions per iter (4 fp16 in one LDG.E.64 vs 4 LDG.E.16s).

What we learned

Lesson 4: occupancy is a means, not an end. Phase 1 v0 was at full occupancy (48 warps/SM) and was slow. v3 is at 33% and is fast. The question isn't "how many warps fit?" — it's *"are we using them for something the kernel actually waits on?"*

For dependency-chain-bound kernels, the answer is no — they wait on chain dependencies, not on parallel memory ops. So trading occupancy for shorter chain is a win.

> **Checkpoint 3.5** - In v3, how many threads per block? How many warps? - Why doesn't v3 need any `__syncthreads()`? - State the v3 trade-off in one sentence, using §2.5's vocabulary.

3.6 v4 — split-K (FlashDecoding) — regression

The hypothesis

v3 launches 256 blocks (8 batch × 32 heads). At 33% occupancy with some hardware-fit-math, it uses about 16 SMs out of 128. **Only 12% of the GPU is busy.**

The textbook fix for grid under-utilization is **split-K** (called FlashDecoding in the attention context): split each (batch, head) pair's work across multiple blocks along the KV sequence dimension. 8× split-K gives 8× more blocks (2048 instead of 256), nominally filling the GPU.

Each split-block handles `seqlen_kv / K_SPLIT` j-positions and produces a partial `(m, l, o_acc)`. A small second kernel combines the K_SPLIT partials into the final output.

Why it should work — in theory

If bandwidth or compute is the ceiling, more SMs busy means more total throughput. Going from 16 → 128 SMs busy should help dramatically.

Why it didn't work — in practice

Measured: 0.802 ms (vs v3's 0.713 ms). 0.89× — regression.

The diagnosis: v3 wasn't actually grid-bound. At 16 SMs busy with ~24 blocks/SM, the *per-SM bandwidth* was the ceiling. Each SM was already at its individual L1/L2 throughput limit reading K and V from the cache.

Adding more SMs doesn't help when **the shared resource is the ceiling**. Going from 16 → 86 SMs means each SM now has fewer blocks (more SMs, same total) and the per-SM bandwidth pressure drops — but the *total* bandwidth across all SMs is still limited by HBM and L2 caps, which were not the ceiling before.

Meanwhile, split-K added: - A second kernel launch (~10 μs). - Scratch memory allocation for partials (~5 μs). - The combine kernel's own work. - Extra HBM writes (partials) + reads (in combine).

Net: ~250 μs of fixed overhead.

What we learned

Lesson 5 (load-bearing): don't add parallelism where the bottleneck isn't compute or per-SM bandwidth. Adding SMs busy is only a win if the *aggregate* of (more SMs × per-SM-throughput) gives more *throughput on the bottlenecked resource*. If the bottleneck is the per-iter chain (which doesn't speed up with more SMs), more SMs is pure overhead.

This is symmetric to lesson 3 (chain trumps sync): both say *attack the actual bottleneck, not the shape that resembles a textbook one*.

Decision

The v4 split-K code was committed for reference (commit `f904aae`), then reverted in `e39c97f`. `main` stays on v3.

> **Checkpoint 3.6** - In §2.8's framework, was v3 bandwidth-bound or chain-bound? What evidence supports your answer? - Why does split-K help in some workloads but hurt in v3's? - When *would* you reach for split-K? (Hint: think about per-SM resource pressure.)

3.7 v5 — cp.async double-buffering — regression

The hypothesis

v3 hides V latency via the prefetch trick (load V at the top of the iter, use it at the bottom). But the prefetch is just *a hint to nvcc*: the actual load still goes through the regular memory path, and we have no control over how many loads are in flight.

`cp.async` (Ampere+) lets you issue asynchronous global → shared memory copies. The thread continues executing while the copy proceeds in the background. With double-buffering, you can have one tile of K/V being copied while the previous tile is being consumed — explicit prefetching with controlled depth.

In theory: **deeper prefetch pipeline → more in-flight loads → better memory latency hiding.**

Why it didn't work

Measured: 0.760 ms (vs v3's 0.713 ms). 0.94× — regression.

The problem: - `cp.async` writes to *shared memory*, not registers. v3 went `HBM → register → use`. v5 goes `HBM → shmem → register → use`. The extra shmem hop costs cycles on every iter (shmem write + shmem read). - v3's prefetch was already capturing most of the available latency hiding through nvcc's compiler scheduling. There wasn't much room for explicit pipelining to add on top.

Net: paid the shmem-hop cost (~50 µs across 4096 iters), gained ~nothing in latency hiding.

What we learned

`cp.async` is the right tool when: - Per-iter compute is heavy enough to amortize the shmem hop. (e.g., prefill attention with Tensor Cores, big GEMMs.) - Compiler-scheduled prefetching can't capture enough latency hiding.

For decode attention v3, neither applies. The per-iter chain is short and nvcc's prefetch is already good enough.

Decision

v5 also committed (`78a28ff`) and reverted (`4254ce2`). `main` stays on v3.

> **Checkpoint 3.7** - Why does `cp.async` write to shared memory, not registers? - What's the structural difference between v1's prefetch fix and v5's cp.async approach? Why does one help and the other hurt? - State the conditions under which `cp.async` would help, in one sentence each.

3.8 The shape of the journey

Step	Latency	BW	Lesson
v0 naive two-pass	1.669 ms	80 GB/s	

Step	Latency	BW	Lesson
			The chain has 2 syncs per `j`. Baseline.
v1 naive (regression)	2.078 ms	65 GB/s	V load inside the sync = exposed latency. Wrong fix attempted.
v1 prefetch (fix)	1.637 ms	82 GB/s	Hoist V load above the sync manually.
v2 single-sync reduce	1.069 ms	126 GB/s	Drop one sync via double-buffer + redundant cross-warp reduce. Shmem hop removal stacks.
v3 vec + 1-warp	**0.713 ms**	**189 GB/s**	Trade occupancy for chain. Won big.
v4 split-K (regression)	0.802 ms	167 GB/s	More SMs doesn't help when per-SM bandwidth is the ceiling.
v5 cp.async (regression)	0.760 ms	177 GB/s	Explicit async loads cost more than nvcc's prefetch already gives.

3.9 Five lessons to take with you

1. **SIMT-parallel ALU work is essentially free.** When warps would otherwise wait at a sync, having them do identical scalar work is free. Moving work to one lane to "save" it often makes things worse (v1 wrong-hypothesis detour).

2. **`__syncthreads()` is a load barrier.** nvcc won't hoist a memory load above it. If a load is on the per-iter critical path after a sync, its latency serialises with the chain. Move the load manually above the sync (v1 prefetch fix).

3. **Removing a shmem hop can dwarf removing a sync.** v2 saved ~570 μs from a change predicted to save only ~150 μs. The chain shortened in more ways than just the visible barrier.

4. **Occupancy is a means, not an end.** v3 traded full occupancy for single-warp blocks and won 1.5×. The lost warps were idle at sync barriers in v2, not doing useful latency hiding.

5. **Don't add parallelism where the bottleneck isn't compute.** v4 split-K added 5× more busy SMs but each SM was still at its per-iter chain limit. More SMs ≠ more throughput when the chain is the ceiling. Same lesson for v5: don't fix latency hiding when it's not the bottleneck.

3.10 Closing thoughts

Part 3's central trick is to apply **the same framework** at every step:

1. *Where's the bottleneck?* Use §2.8 (bandwidth / compute / chain).
2. *What lever shortens that specifically?* §2.7 (the chain), or §2.3 (memory hierarchy), or §2.5 (occupancy).
3. *Predict the magnitude.* If your prediction is off by 4×, you're missing something — go look at the chain again.

We landed at **v3: 0.713 ms / 189 GB/s — 1.91× over PyTorch SDPA** through five iterations and two reverts. Every step in the table above is on a separate branch (`v0`, `v1-naive`, `v1`, `v2`, `v3`, `v4`, `v5`) — check them out, rebuild, run the bench, and feel the numbers in your hands.

Part 4 (KV-cache compression) and Part 5 (cross-cutting lessons) build on these foundations. By the end of the book, you should be able to look at any new GPU kernel and walk through Parts 2's diagnostic and Part 3's pattern-matching without referring back.

Ready for Part 4? Part 4 covers KV-cache compression: the math of symmetric integer quantization, why per-channel K + per-token V (KIVI) is the right recipe, scale-folding via linearity in the INT8 kernel, and the structural per-group K-scale trick that makes INT4 actually *faster* than fp16 (where INT8 was a tie).

Before continuing, work checkpoints 3.2 (online softmax) and 3.4 (v2's sync removal) cold. They're the math/structural pair you'll extend in Part 4.

Part 4 — KV-cache compression

Phase 1 made the *compute* fast. Phase 2 (the subject of this part) shrinks the *memory*: replace fp16 KV cache storage with integer storage, dequantizing on the fly inside the attention kernel.

The arithmetic in Part 4 is simpler than Part 3's. The interesting work is **structural** — which axis the scale shares values across, how the kernel exploits that structure for both *quality* and *latency*, and how to validate the choice at the model level (not just the kernel level).

By the end of Part 4 you should be able to:

1. Derive symmetric integer quantization (scales, qmax, rounding) from scratch.
2. Explain why K and V want *different* quantization recipes (KIVI).
3. Recognise when a structural trick will shorten the per-iter chain — and when it just shrinks bytes without changing the chain.
4. Set up a model-level perplexity measurement for a new quantization scheme via the `F.scaled_dot_product_attention`-patching trick.

4.1 The KV-cache problem, deeper

From §1.6: KV cache stores fp16 K and V across decode steps. Per token across all layers of Llama 3 8B:

```
2 · n_layers · n_kv_heads · head_dim · sizeof(fp16)
= 2 · 32 · 8 · 128 · 2
= 131,072 B = 128 KiB / token
```

Scale that up to a chat-serving workload of `batch=32, seqlen=8192`: **~34 GiB.** Doesn't fit in a 24 GB RTX 4090 alongside the model weights, never mind a larger model.

So the KV cache caps two things in practice: - **Memory**: the number of concurrent users × their context length. - **Bandwidth**: every decode step reads the full cache. At `batch=8, seqlen_kv=4096`: 128 MiB of K + V *per decode step*.

Phase 2's question: **can we shrink it without breaking quality?**

The implicit second question is whether shrinking it also speeds up the attention kernel. Part 1 said decode attention "should be" memory-bound; if true, half the bytes ≈ half the time. Part 3 showed v3 was actually chain-bound (19% of HBM peak). So the latency answer isn't obvious before measuring.

> **Checkpoint 4.1** Why does the KV cache scale with `seqlen` but the weights don't? What does this imply for choosing between weight quantization (Phase 3) and KV-cache quantization (Phase 2) in terms of which problem they solve?

4.2 Symmetric integer quantization, from scratch

The simplest quantization scheme. Two parameters:

- **Bit width**: 8 (INT8) or 4 (INT4). Determines `qmax = 2^(bits-1) - 1`, the largest representable signed integer. INT8 has `qmax = 127`; INT4 has `qmax = 7`.
- **Scale axis**: which group of values shares a single scale.

Given a set of fp16 values `x[i]` sharing one scale:

```
absmax = max(|x[i]|)
scale  = absmax / qmax            // fp16
q[i]   = round(x[i] / scale),     // map to int, clamp to [-qmax, qmax]
         clamped to [-qmax, qmax]
```

Dequantize:

```
x_hat[i] = q[i] · scale
```

Properties: - `|x_hat[i] - x[i]| ≤ scale / 2` (rounding error is bounded by half a step). - Per-element error vs `absmax` is `≤ 1 / (2 · qmax)`. INT8: ~0.4%. INT4: ~7%.

The outlier problem

Per-element error is *bounded by absmax*. But what if some elements are much smaller than absmax?

Example: `x = [-0.05, 0.05, -0.03, 0.04, 2.0]`. The outlier `2.0` sets `absmax = 2.0`. INT8 scale = `2.0 / 127 ≈ 0.0157`.

The "normal" value `0.05` quantizes to `round(0.05 / 0.0157) = 3`, then dequantizes to `3 · 0.0157 = 0.0471`. Error 0.0029, but as a *fraction of 0.05*, that's **6%**. The outlier crushed the resolution for everyone else.

This is the **outlier problem**. The scheme is forced to use big steps to cover the outlier, and the steps are coarse for non-outlier values.

The fix is to *not* share a scale across values with such different magnitudes. Which brings us to §4.3.

> **Checkpoint 4.2** - For INT4 with `qmax = 7`, what's the maximum per-element error as a fraction of `absmax`? - In the example above, what would the relative error on `0.05` be if we used INT4 instead of INT8? - The error bound `scale / 2` is *per-element*. Why doesn't that mean a kernel using these dequantized values has bounded relative error on the final output? (Hint: think about the dot product.)

4.3 Per-token vs per-channel: the structural choice

A K (or V) cache tensor has shape `[batch, n_kv_heads, seqlen, head_dim]`. For symmetric quantization, **which axis does the absmax reduce over** — i.e., which values share a scale?

Two options:

Per-token: one scale per `(batch, kv_head, token)`. The absmax reduces over `head_dim` (128 values per scale). Scale shape: `[batch, n_kv_heads, seqlen]`.

Per-channel (groupwise along seqlen): one scale per `(batch, kv_head, group, channel)`. Groups partition the seqlen axis in chunks of `group_size = 32` tokens. The absmax reduces over the `group_size` tokens within the group, *per channel*. Scale shape: `[batch, n_kv_heads, n_groups, head_dim]`.

Trade-off:

Scheme	Values per scale	Storage of scales
Per-token	128 (one head_dim)	1 fp16 per token

Scheme	Values per scale	Storage of scales
Per-channel groupwise (g=32)	32 (one group of tokens)	128 fp16 per group

Both have small scale storage. The real question is *quality* — which direction has more variation in magnitudes, and therefore *which one needs the per-axis flexibility*?

If magnitudes vary mostly **between tokens** (and within a token, head_dim values are similar): per-token is right.

If magnitudes vary mostly **between channels** (and within a channel, many tokens have similar magnitudes): per-channel is right.

This is an empirical question about the data, not a math question.

> **Checkpoint 4.3** - For per-channel groupwise quantization with `group_size = 32` and `seqlen = 4096`, how many groups are there? - You're told that for some tensor, the magnitudes vary *both* between tokens AND between channels. Which scheme is better, or should you do both (per-token-per-channel)?

4.4 KIVI's two findings

KIVI (Liu et al. 2024) measured the K and V activations of real LLMs and found:

1. **K has persistent per-channel outliers.** Certain `head_dim` positions consistently have much larger magnitudes than others, across all tokens. These outlier channels stay outlier-ish across the whole sequence.
2. **V doesn't.** V's magnitudes are roughly uniform across `head_dim`.

The recipe that falls out of these two measurements:

Tensor	Scale axis	Reason
K	Per-channel groupwise	Outlier channels each get their own scale; non-

Tensor	Scale axis	Reason
		outlier channels aren't crushed.
V	Per-token	head_dim values within a token are uniform; one scale per token wastes nothing.

At **INT8**, this distinction barely matters — INT8's resolution (`qmax = 127`) is fine enough that either scheme works. The K-outlier crushing of non-outlier channels (from §4.2's analysis) costs maybe a percent or two of element-level accuracy, which softmax washes out.

At **INT4** (`qmax = 7`), the K-outlier crushing becomes catastrophic. Per-token K at INT4 leaves only a few representable values for the non-outlier channels, and the model quality degrades sharply.

KIVI's contribution: **at INT4, per-channel K is the difference between "essentially indistinguishable" and "noticeable degradation"**. We measured this directly in Phase 2d (see §4.10).

> **Checkpoint 4.4** If we tried per-channel V instead of per-token V (both at INT4), would you expect quality to be better, worse, or about the same as KIVI's per-token V? Why?

4.5 Phase 2b: INT8 implementation

Storage layout: - `K_q : [batch, n_kv_heads, seqlen, head_dim]` int8. - `K_scale : [batch, n_kv_heads, seqlen]` fp16. One scale per token. - Same for `V_q` and `V_scale`.

Compared to fp16 KV (2 B per element), this is 1 B per element + ~0.78% scale overhead — **0.51× the fp16 size**.

The CUDA kernel reads `K_q` (int8) and `K_scale` (fp16) directly from HBM, dequantizes in registers, and proceeds with v3's attention math.

Naive dequant: per-lane multiply

The straightforward approach:

```
for j in 0..seqlen_kv:
    // 4 ints per thread, plus per-token scale
    k_int[0..3] = K_q[j, tid*4..tid*4+3]
    v_int[0..3] = V_q[j, tid*4..tid*4+3]
    k_s         = K_scale[j]       // fp16 → float
    v_s         = V_scale[j]

    // Per-lane dequant (4 multiplies)
    k_v[0..3] = k_int[0..3] · k_s
    v_v[0..3] = v_int[0..3] · v_s

    // Then v3's attention body:
    partial = q.x·k_v[0] + q.y·k_v[1] + q.z·k_v[2] + q.w·k_v[3]
    ... warp reduce + softmax + V FMA ...
```

This works. But there's an elegance available.

4.6 The scale-folding linearity trick

Look at the K dot product:

```
q · k = Σ_d q[d] · k[d]
      = Σ_d q[d] · (k_int[d] · k_s)
      = k_s · Σ_d q[d] · k_int[d]      // pull the scalar k_s out
```

The K scale is *one value per token* — a scalar, the same for all `d`. We can factor it out of the sum. Compute the dot product on int-valued `k_int`, do one multiply by `k_s` at the end.

For V, the FMA is:

```
o[d] += p_j · v[d]
      = p_j · (v_int[d] · v_s)
      = (p_j · v_s) · v_int[d]
```

Same trick: fold `v_s` into `p_j` once per iter, then FMA with `v_int` directly.

The refactored inner loop:

```
for j in 0..seqlen_kv:
    k_int = K_q[j, tid*4..tid*4+3]
    v_int = V_q[j, tid*4..tid*4+3]
    k_s   = K_scale[j]
    v_s   = V_scale[j]
```

```
    partial    = q.x·k_int[0] + q.y·k_int[1] + q.z·k_int[2] +
q.w·k_int[3]
    partial    = warp_reduce_sum(partial)
    s_j        = partial · k_s · softmax_scale       // fold K
scale here

    m_new      = max(m_state, s_j)
    α          = exp(m_state - m_new)
    p_j        = exp(s_j - m_new)
    p_j_scaled = p_j · v_s                            // fold V
scale here

    o_state.x += p_j_scaled · v_int[0]
    o_state.y += p_j_scaled · v_int[1]
    ... (with α-rescale on o_state, l_state, m_state) ...
```

Saves 8 multiplies per iter vs the naive per-lane dequant (4 K dequant + 4 V dequant multiplies, replaced by 1 K-side fold + 1 V-side fold).

This linearity-folding trick depends on the scale being a *scalar* across the axis you're summing over. **It works for per-token scales. It would NOT work for per-channel scales** (§4.8 explains).

> **Checkpoint 4.6** - Re-derive the linearity factoring for K from scratch. Where does the scalar property of `k_s` get used? - For the V FMA fold, why is `p_j` also a scalar (across `d`)? - In Phase 1 v3 (no quantization), is there a "scale" the same linearity could fold? (Hint: yes — what's `softmax_scale`?)

4.7 Why INT8 tied with v3 — the chain doesn't move

INT8 attention measured: **0.713 ms — exactly tied with v3 fp16.**

Naively this is surprising: INT8 reads *half* the KV bytes, so if v3 were bandwidth-bound, INT8 should be ~2× faster.

Using Part 2's framework:

Was v3 bandwidth-bound? §3.5 showed v3 hits 189 GB/s of 1008 GB/s peak (19%). Not at peak. INT8 attention hits 96 GB/s (half the bytes in the same time). Also not at peak. **Neither is bandwidth-bound.**

So what is the bottleneck? The per-iter chain (from §3.5):

```
load K → warp_reduce_sum → softmax (max + 2× exp) → V FMA → next iter
```

This shape is *the same* in v3 and INT8. The K load is smaller in INT8, but the load is one step in the chain; making it shorter (a few cycles) doesn't change the chain's total latency much. The warp_reduce_sum, softmax, and FMA are unchanged. Same chain = same per-iter time = same wall-clock.

The lesson is fundamental, and recurs: changing the bytes without changing the per-iter chain doesn't change latency on a chain-bound kernel. Same as Phase 1 v4 (split-K), Phase 1 v5 (cp.async). Different levers (more SMs, more in-flight loads, smaller bytes) — same diagnosis (chain didn't move, so latency didn't either).

But INT8 is still a **memory** win. 0.51× of fp16 KV means roughly: - ~2× longer context in the same VRAM, or - ~2× larger batch.

And Δppl on WikiText-2 is +0.0008 — essentially lossless (§4.10).

So INT8 KV is a no-brainer drop-in for serving stacks: same speed, half the memory, no quality loss. The wins come from infrastructure (more users / longer context) not the kernel-step latency.

> **Checkpoint 4.7** - For a kernel that's *truly* bandwidth-bound (≈ 1008 GB/s on the 4090), would INT8 KV speed it up? By roughly how much? - State, in one sentence, the lesson that recurs across Phase 1 v4, Phase 1 v5, and Phase 2b.

4.8 Phase 2c: INT4 KIVI

Storage layout (packed): - `K_q : [batch, n_kv_heads, seqlen, head_dim/2]` int8, 2 nibbles per byte. - `K_scale : [batch, n_kv_heads, n_groups, head_dim]` fp16. *Per- channel groupwise* — one scale per `(group, channel)`. - `V_q : [batch, n_kv_heads, seqlen, head_dim/2]` int8, packed. - `V_scale : [batch, n_kv_heads, seqlen]` fp16. *Per-token*.

This is **0.27× of fp16** — 4× smaller storage on the values, plus small per-group K-scale overhead and tiny V-scale overhead.

The kernel's job: read packed bytes, unpack (one int8 byte → 2 signed 4-bit values via the shift-trick from Part 3 / Phase 3 work), dequantize with the right scale, do attention.

Why the §4.6 trick doesn't apply directly

The K dot product, with per-channel K scales:

```
q · k = Σ_d q[d] · k[d]
      = Σ_d q[d] · (k_int[d] · k_scale[g, d])    // k_scale indexed by d!
```

Now `k_scale[g, d]` is *inside* the sum (depends on `d`), so we can't factor it out. Per-lane dequant is back — 4 multiplies per thread per iter just for K dequantization.

That's bad. We expected INT4 to win on memory; this kernel-cost story makes the latency outlook worse, not better.

The structural trick: per-group pre-scaling

But: `k_scale[g, d]` is **constant within a group**. For all iters `j` such that `g(j) = g`, the K scale is the same. So instead of loading the K scale per iter, we can:

1. **Outer-loop over groups** `g = 0..n_groups - 1`.
2. **Per-group preamble**: load all 4 K scales for this thread's lanes into registers. Then *pre-scale q*:

   ```
   q_scaled.x = q.x · k_scale[g, tid*4]
   q_scaled.y = q.y · k_scale[g, tid*4+1]
   q_scaled.z = q.z · k_scale[g, tid*4+2]
   q_scaled.w = q.w · k_scale[g, tid*4+3]
   ```

   That's 4 multiplies, once per group.
3. **Inner loop over tokens within the group**: the K dot product is now

   ```
   q_scaled · k_int = Σ_d q_scaled[d] · k_int[d]
                    = Σ_d (q[d] · k_scale[g, d]) · k_int[d]
   ```

```
                = Σ_d q[d] · k_int[d] · k_scale[g, d]
                = q · (k_int · k_scale) = q · k          ✓
```

Correct, but now the inner loop is just `int_values` and pre-scaled `q_scaled`. **No per-iter K-scale load. No per-iter K dequant.**

The amortized cost of the per-group preamble: 4 multiplies per group ÷ `group_size = 32` iters per group = **0.125 multiplies per iter** for the K dequant. Down from 4 per iter (naive per-lane).

For V (per-token), the scale-folding §4.6 trick still works: `p_j · v_s` folds into the FMA coefficient.

The full inner loop:

```
for g in 0..n_groups:
    // Per-group preamble: load 4 K scales, pre-scale q (in
registers)
    k_scale_v = K_scale[g, tid*4..tid*4+3]        // 4 fp16 → 
float
    q_scaled.x = q.x · k_scale_v.x
    q_scaled.y = q.y · k_scale_v.y
    q_scaled.z = q.z · k_scale_v.z
    q_scaled.w = q.w · k_scale_v.w

    for j in g·group_size..(g+1)·group_size:
        // Load packed int4 K and V (one uint16 per thread → 4
nibbles)
        k_int[0..3] = unpack(K_q[j, tid*2..tid*2+1])
        v_int[0..3] = unpack(V_q[j, tid*2..tid*2+1])
        v_s         = V_scale[j]                   // per-token V
scale

        partial    = q_scaled.x·k_int[0] + q_scaled.y·k_int[1]
+ ...
        partial    = warp_reduce_sum(partial)
        s_j        = partial · softmax_scale       // K scale
already folded!

        m_new      = max(m_state, s_j)
        α          = exp(m_state - m_new)
        p_j        = exp(s_j - m_new)
        p_j_scaled = p_j · v_s
```

```
        o_state += p_j_scaled · v_int              // FMA on ints
        // ... rest of softmax update ...
```

The per-iter chain has lost the K-scale load (every iter). It also has shorter K and V loads (uint16 vs uint32) because of the packed format.

> **Checkpoint 4.8** - Why was it crucial that `k_scale[g, d]` is constant within a group for the pre-scale-q trick to work? - Per-iter, how many multiplies for K dequant in the naive INT4 kernel vs the per-group-pre-scaled version? Amortise. - The trick wouldn't work for *per-token* K scales (one per token). Why? (Hint: how often does the scale change?)

4.9 Why INT4 sped up where INT8 tied

INT4 KIVI measured: **0.554 ms — 1.29× faster than v3 / INT8.**

Apply Part 2's framework: did the **chain** change? Let's compare inner loops:

Element	INT8 per-token	INT4 KIVI
Load K_q	4 B (LDG.E.32)	2 B (LDG.E.16)
Load K_scale	**1× fp16 / iter**	**0** (cached per-group)
Load V_q	4 B	2 B
Load V_scale	1× fp16 / iter	1× fp16 / iter
K dequant in inner loop	"fold via dot" (1 multiply post-reduce)	none (pre-scaled q)
V dequant fold (p_j scale)	1 multiply	1 multiply
Warp reduce + softmax + FMA	identical	identical

The big change: **INT4 KIVI has one fewer load on the per-iter critical path** (the K scale). Also two of the loads (K_q and V_q) are half the size.

Loads on the chain are *latency-bearing* (the value has to come back before the dot product can complete). Removing a load shortens the chain proportionally to that load's contribution to the chain — likely ~10-30 cycles per iter at HBM/L2 hit times. Times 4096 iters times many parallel blocks: real wall-clock.

So INT4 KIVI moves the chain in a way INT8 didn't. INT8 just shrank bytes; INT4 KIVI shrank bytes **and** removed a load from the inner loop.

The lesson, made explicit: byte-shrinking changes alone don't help chain-bound kernels. Byte-shrinking changes that *also* lift loads out of the inner loop do.

And so we land at: **INT4 KIVI is 0.27× memory and 1.29× latency over fp16 v3.** Wins both axes.

> **Checkpoint 4.9** - Explain in one sentence why INT8 tied with v3 but INT4 KIVI beat it, using the word "chain." - If we had INT4 with *per-token* K scales (no KIVI), would the per-group pre-scale-q trick apply? Would the kernel be faster than INT8?

4.10 Phase 2d: measuring perplexity

We've built kernels. The kernel-level error vs the fp16 reference is small (max abs diff: 1.1e-3 for INT8, 2.3e-2 for INT4 — see §4.5, §4.8). But the **model-level** quality is what matters: when Llama 3 generates text reading the compressed cache, how much worse is the output?

The standard metric is **perplexity** on a held-out dataset (we use WikiText-2 test, 131,008 tokens across 64 chunks of 2048):

```
perplexity = exp(- 1/N · Σ log p_model(token_i | tokens_<i))
```

Lower is better. The Phase 2 success criteria from `docs/02`: - INT8: Δppl < 0.2 (essentially indistinguishable threshold). - INT4 KIVI: Δppl < 0.5 (acceptable target).

The patch-`F.scaled_dot_product_attention` trick

To measure this, we need Llama 3 8B's attention to *use the quantized KV cache* during a 64-chunk forward pass.

A real model integration (Phase 4 work) needs a custom attention class that allocates an INT8 or INT4 KV cache. Substantial lift.

A **clever proxy** that's mathematically equivalent: patch the `torch.nn.functional.scaled_dot_product_attention` entry point. The Llama attention layer calls `F.scaled_dot_product_attention(Q, K, V)` internally. We intercept that call, quantize the `K` and `V` inputs with our reference, dequantize them back to fp16, and pass the *noisy fp16 values* into the original `F.sdpa`.

```
def patched_sdpa(query, key, value, ...):
    # Quantize K with KIVI's per-channel groupwise (or per-token
        int8)
    key_q   = quantize_kivi_int4(key)
    value_q = quantize_per_token_int4(value)
    # Dequantize back
    key_back   = dequantize(key_q)
    value_back = dequantize(value_q)
    # Call original SDPA with the round-tripped (noisy) K, V
    return original_sdpa(query, key_back, value_back, ...)

torch.nn.functional.scaled_dot_product_attention = patched_sdpa
```

The model sees fp16 K, V at the SDPA boundary — but those are *the same fp16 values* it would see if the real INT4 KV cache were in use and our CUDA kernel were doing the dequant. The kernels and the patched sdpa implement the same math:

```
attention_output = softmax(Q · dequant(K_q)^T / √d) · dequant(V_q)
```

They differ in *where* the dequant happens (fused inside the kernel vs upfront in patched sdpa), not in the values flowing through softmax.

Results

Mode	ppl	Δppl from fp16	Threshold	Verdict
fp16 baseline	7.055	—	—	—
INT8 per-token K, V	7.056	+0.0008	< 0.2	✅
INT4 per-token K, V (naive, no KIVI)	7.517	+0.462	—	(KIVI comparator)
INT4 KIVI (per-channel K, per-token V)	**7.252**	**+0.196**	**< 0.5**	✅

Two key results:

1. **INT8 is essentially lossless** (Δppl = +0.0008 on 131k tokens). The K-outlier crushing predicted by §4.2 just isn't a big deal at INT8's resolution.

2. **At INT4, KIVI matters enormously.** Naive INT4 per-token K → Δppl +0.462 (would barely pass the threshold). KIVI per-channel K → Δppl +0.196 (passes with margin). **KIVI's contribution: 2.36× improvement in quality.**

The §4.4 prediction — that per-channel K matters at INT4 specifically — is *measured*.

> **Checkpoint 4.10** - Why does the patched SDPA give the *same numbers* you'd get from running the actual INT4 KIVI CUDA kernel through Llama? (Hint: what's the math both paths implement?) - At INT8, naive per-token K (not KIVI) would presumably also work. Did we measure it? Why or why not? (Hint: design choice.)

4.11 The kernel-level vs model-level accuracy gap

Phase 2c reported INT4 KIVI's kernel-level *mean relative error* on random gaussian K, V as **42%**. Yet on a real Llama model, Δppl is +0.196 — that's about **2.78% relative perplexity change**. A gap of ~15×.

What gives?

1. **Softmax max-subtraction is forgiving.** Attention's output is `Σ_j p_j · V[j]`. The softmax assigns most of the weight to a few high-score tokens; the rest have tiny `p_j` and contribute negligibly. So per-element errors on `V` mostly cancel — only the high-`p_j` tokens shape the output. And of those, the *ranking* of scores matters more than absolute magnitudes. Small quantization noise rarely flips ranking.

2. **Real LLM activations have structure.** Random gaussian K, V is the *worst case* — every channel has comparable magnitude, no sparsity to exploit. Real Llama K activations have persistent per-channel outliers (the entire reason KIVI uses per-channel K). With

KIVI's per-channel scales, outliers get their own scale and the non-outlier channels are quantized fine. This is exactly the scenario KIVI is *designed for*.

3. **The model can absorb noise.** Llama 3 8B has 32 transformer layers. Each layer's LayerNorm + subsequent attention/MLP can smooth out small perturbations. A 1% error in one layer's activations rarely propagates to a 1% error in the next layer's output.

> **Lesson 4.A**: kernel-level error on random data is a *smoke test*, not a quality verdict. Always validate at the model level on real activations.

This applies to any quantization or numerically-different kernel — not just KV cache.

4.12 Summary of Part 4

Step	Storage	Latency	Δppl WikiText-2	Lesson
fp16 KV (v3)	128.0 MiB	0.713 ms	0	baseline
INT8 per-token	65.0 MiB (0.51×)	0.713 ms (tied)	+0.0008 (lossless)	bytes ↓, chain same → latency same; memory free
INT4 KIVI	34.5 MiB (0.27×)	**0.554 ms (1.29× faster)**	**+0.196 (within 0.5)**	per-group scale lift + smaller loads shorten chain
INT4 per-token (naive)	similar	—	+0.462	KIVI's per-channel K is worth

Step	Storage	Latency	Δppl WikiText-2	Lesson
				2.36× quality

4.13 Five lessons to carry forward

1. **The outlier problem motivates the scale axis.** Symmetric quantization with a shared scale is forced to use coarse steps if any value in the group is an outlier. The non-outlier values get crushed. *Pick the scale axis where magnitudes vary least.*

2. **K and V want different recipes.** K has per-channel outliers; V doesn't. Per-channel K + per-token V (KIVI) is not aesthetic — it's what the data wants.

3. **The scale-folding linearity trick works only for scalar scales.** For per-token scales (one fp16 across head_dim), the scale factors out of the dot product. For per-channel scales, it doesn't — but you can fold *q* instead, once per group.

4. **Byte-shrinking alone doesn't speed up chain-bound kernels.** This is the deepest lesson of Phase 2 — and the same lesson as Phase 1 v4/v5. INT8 KV shrank bytes 2× but tied with v3 because the chain didn't move. INT4 KIVI shrank bytes 4× *and* lifted K-scale loads out of the inner loop — the chain moved, and latency dropped 1.29×.

5. **Kernel-level error on synthetic data overstates model-level quality cost.** INT4 KIVI's 42% kernel-level rel err became 2.78% model-level Δppl. Softmax max-subtraction + real-activation structure + multi-layer noise absorption all save you. Don't gate quantization decisions on synthetic-data error alone.

Closing

We're 17,000 words into the book. Parts 1–4 cover the *kernel journeys*; what remains is the **cross-cutting workflow** — how to apply Parts 1–4's framework to a *new* kernel you've never seen. That's Part 5.

Before continuing, work checkpoints 4.4 (KIVI K/V asymmetry), 4.8 (per-group pre-scale trick), and 4.9 (why INT4 moved the chain) cold. Those three are the load-bearing pair-ups for Part 5 patterns.

Ready for Part 5? Part 5 (the closing part) pulls the cross- cutting lessons from Phases 1-3 into a single working playbook: when you sit down with a new kernel, what questions do you ask first, what optimizations do you try first, and what traps recur across kernel families.

Part 5 — W4A16 GEMM and the cross-cutting workflow

This part has two halves. **Section A** walks Phase 3 — the W4A16 quantized matmul, the third major kernel of the project. **Section B** distills Parts 1–4 + Phase 3 into a working playbook: when you sit down with a new kernel, what to ask, what to measure, what to try.

After Part 5 you should be able to:

1. Walk the v0 → 3c progression of W4A16 GEMM from memory.
2. Recognise *which* of the recurring traps (redundant-work, more-SMs, more-async-loads, smaller-bytes) is being set up in a given proposal.
3. Apply the six-question playbook to any new GPU kernel you encounter.

Section A — Phase 3: W4A16 quantized matmul

5.1 Why weights are different from KV

Phase 2 compressed the *KV cache*: data that grows with sequence length and gets cached across decode steps. Phase 3 compresses the *model weights*: a different beast.

Property	KV cache (Phase 2)	Weights (Phase 3)
Where the data lives	HBM, grows with `seqlen`	HBM, static
When it's quantized	At decode time (streaming)	Once, offline

Property	KV cache (Phase 2)	Weights (Phase 3)
Size in Llama 3 8B	~128 KiB / token / all layers	~16 GB total (fp16)
Read pattern	Read full cache per decode	Read whole W per linear
Reuse	Once per decode step	Once per decode step

Weights are HUGE: Llama 3 8B has ~7B parameters in linear layers (QKV projections, output projection, MLP up/gate/down). At fp16 that's ~14 GB — *bigger than the KV cache* at typical context lengths. And every decode step reads the entire weight tensor of every linear layer.

So for decode latency, **weight HBM traffic is dominant**. The same "memory-bound thesis" from Phase 2 applies, just to a different tensor. INT4 weights cut weight HBM bytes 4×; that's potentially up to 4× faster decode for every linear layer.

The kernel is called a **W4A16 GEMM**: W=weights at 4 bits, A= activations at 16 bits (fp16), and "GEMM" is the matmul family. It takes fp16 activations and INT4 weights, produces fp16 outputs.

> **Checkpoint 5.1** - Why is "quantize once offline" easier than "quantize every decode step"? What kind of quality loss does each scheme tolerate? - For Llama 3 8B at fp16, what's larger: the weights or the KV cache at `batch=8, seqlen_kv=4096`? At `batch=32, seqlen=8192`?

5.2 W4A16 quantization recipe

This is *exactly* KIVI K's recipe, applied to a weight matrix. Same symmetric integer math from §4.2, same per-channel groupwise structure from §4.4.

Weight `W : [K, N]` fp16 (where K = in_features, N = out_features in the matmul `out[M, N] = act[M, K] · W[K, N]`).

Quantization scheme: - **Per-channel groupwise**: one scale per `(group along K, output channel along N)`. Group size = 128 K-positions per group. - **4-bit signed**: `qmax = 7`. Values in `[-7, 7]`. - **Packed**: 8 nibbles per `uint32` along K. Storage shape `[K/8, N]` int32. Bit `i·4..i·4+3` of position `(k_pack, n)` represents K position `k_pack·8 + i`.

Storage size: - fp16 W: `K · N · 2 bytes`. - INT4 W: `K · N / 2 bytes` for packed values, plus `n_groups · N · 2 bytes` for fp16 scales. With group_size=128: scales overhead is `≈ 1/128 · fp16 size`. Total: ~0.258× of fp16.

For Llama 3 8B linear layers: - attn QKV/O (K=4096, N=4096): 32 MiB fp16 → 8.25 MiB W4A16. - MLP up/gate (K=4096, N=14336): 112 MiB fp16 → 28.88 MiB W4A16. - MLP down (K=14336, N=4096): 112 MiB fp16 → 28.88 MiB W4A16.

Total Llama 3 8B weight footprint: 14 GB fp16 → ~3.6 GB W4A16. A factor of 4 storage reduction.

> **Checkpoint 5.2** Compare the §4.4 KIVI K recipe to the §5.2 W4A16 recipe. List two ways they're the same and one way they differ.

5.3 The decode-shape GEMM (M=1)

For a decode step, each linear layer multiplies the current token's hidden state (shape `[1, K]`) by the weight matrix (shape `[K, N]`) to produce one row of output (shape `[1, N]`):

```
out[0, n] = Σ_k act[0, k] · W[k, n]      for n in 0..N-1
```

This is technically a GEMM with `M = 1` — but really it's a *gemv* (matrix-vector). Tensor Cores want `M, N, K ≥ 16` for efficient MMA; at `M = 1` they're underused. **The decode GEMM is memory-bound on the weight load**, not compute-bound on the FMAs.

Concrete numbers, MLP up/gate at M=1, fp16: - Weight: `K · N · 2 = 4096 · 14336 · 2 = 112 MiB`. Has to come from HBM (or L2 if cached). - Compute: `K · N · 2 = 117 M FLOPs`. Trivial — would take microseconds at the 4090's ~83 TFLOPS peak.

cuBLAS fp16 measures 0.134 ms here, of which 0.111 ms (83%) is just moving 112 MiB at HBM peak speed. So cuBLAS is *itself* memory-bound on weight traffic. The "Phase 3 thesis" — 4× less weight bytes → potentially 4× faster — fits cleanly.

> **Checkpoint 5.3** - At M=1, why is the GEMM memory-bound rather than compute-bound? - For the MLP up/gate shape, what's the theoretical minimum time to read 112 MiB at HBM peak (1008 GB/s)? - What about at the *L2* level (72 MiB cache, much higher bandwidth)? How does that change the analysis?

5.4 Phase 3b: naive W4A16 kernel

Block geometry (same template as Phase 1 v3): - Grid: `N / BLOCK_N` blocks, where `BLOCK_N = 32` (output columns per block). - Block: 32 threads = 1 warp. Each thread owns one output column `n = block_n_base + tid`.

Inner loop per (m, n):

```
acc = 0
for g in 0..n_groups:
    scale = scales[g, n]                                    // fp16
→ float
    for p in 0..packs_per_group:
        k_pack = g · packs_per_group + p
        w_uint32 = weight_packed[k_pack, n]                  // one
LDG.E.32
        for i in 0..7:                                       //
unrolled
            nibble = sign_extend((w_uint32 << (28 - i·4)) >> 28)
            k = k_pack · 8 + i
            acc += act[m, k] · (nibble · scale)
out[m, n] = (half) acc
```

The shift-trick sign-extends a 4-bit signed value: shift the nibble to bits 28-31, then arithmetic shift right by 28 (which sign-extends because the high bit was the sign bit).

Coalescing: across the 32 threads in the warp at iter `k_pack`, they load `weight_packed[k_pack, n_base..n_base+31]` — 32 contiguous int32 = one 128-byte coalesced warp load.

Result, all M=1 Llama shapes

Shape (K, N)	cuBLAS fp16	3b naive	Speedup
4096 × 4096 (attn)	0.047 ms	0.088 ms	0.53× **(loss)**

Shape (K, N)	cuBLAS fp16	3b naive	Speedup
4096 × 14336 (MLP up)	0.134 ms	**0.084 ms**	**1.59× (win)**
14336 × 4096 (MLP down)	0.133 ms	0.284 ms	0.47× (**loss**)

One **threshold hit** (MLP up/gate by 1.59×) and two losses.

Diagnosing the losses with Part 2's framework

The wins are concentrated where: - N is large (many blocks → enough grid) - K is moderate (4096, not 14336 — shorter per-thread reduction)

The losses correspond to: - **4096 × 4096 (attn)**: only `N/BLOCK_N = 128` blocks. With 128 SMs on the 4090, that's ~1 block per SM. Each block has 1 warp, so each SM has 1 warp resident → severe under-occupation. Way below the ~16-24 warps/SM needed for latency hiding. - **14336 × 4096 (mlp-down)**: K is 14336, so each thread's serial-FMA loop is 14336 iterations long. Even with reasonable block count, the per-thread work is too sequential. The dependency chain through the inner loop is the bottleneck.

So the 3b kernel: - Wins when both grid is large *and* per-thread K is moderate. - Loses when either is too small.

The fix from Part 3 we'd reach for: *shorten the per-iter chain* and *increase per-block parallelism*. That's 3c.

> **Checkpoint 5.4** - For 3b, what's the per-iter chain inside the inner loop (per thread)? - Why does N affect grid utilization but K doesn't (in 3b)? - What lesson from Phase 1 v4 applies here, and which doesn't?

5.5 Phase 3c: K-split across warps + act in shmem

Two structural changes:

Change 1: Multi-warp block with K split.

Block grows from 1 warp to 4 warps (128 threads). The output tile stays the same — 32 columns, owned by one warp's 32 lanes (each lane owns one column).

K is split across the 4 warps: - Warp 0 processes `K/4` of the K-reduction for all 32 columns. - Warp 1 processes the next `K/4`. - … etc.

After the K loop, each thread has its own partial fp32 accumulator. A tiny shmem-based combine sums the 4 partials per column to produce the final output:

```
partials_smem[warp_id · 32 + lane] = local_acc       // each warp
writes 32 partials
__syncthreads()
if warp_id == 0 && lane < 32:                         // warp 0
sums them
    total = Σ_w partials_smem[w · 32 + lane]
    out[block_n_base + lane] = (half) total
```

The block-wide sync is once per output (not per inner iter, like attention had). It's negligible vs the K loop.

Change 2: `act` cached in shared memory.

Each block's 4 warps all need the same K-length activation vector. We cooperatively load `act[0..K)` into shmem at kernel start (128 threads × `K/128` elements each), then read from shmem in the inner loop. Frees L1 for weight traffic.

For K=4096, that's 8 KiB of shmem. For K=14336, 28 KiB. Both well under the 100 KiB/SM cap.

Why this helps each losing shape

Shape (K, N)	3b problem	What 3c fixes
4096 × 4096	128 blocks × 1 warp = 128 warps total	128 blocks × 4 warps = 512 warps → 4× more SMs busy
14336 × 4096	Per-thread K-loop = 14336 iters	K/4 = 3584 iters per thread (each warp gets a quarter)
4096 × 14336	Already winning	K reduced 4× per thread anyway → even faster

Result

Shape (K, N)	cuBLAS fp16	3b naive	3c decode	Speedup over cuBLAS
4096 × 4096 (attn)	0.047 ms	0.088 ms	**0.016 ms**	**2.88×**
4096 × 14336 (MLP up)	0.134 ms	0.084 ms	**0.019 ms**	**6.97×**
14336 × 4096 (MLP down)	0.133 ms	0.284 ms	**0.045 ms**	**2.96×**

All three M=1 shapes clear Phase 3 Target (2-3× over cuBLAS), with MLP up/gate at nearly 7×.

The improvement factorises beautifully:

Shape	3b → 3c improvement
4096 × 4096	5.4× (4× more warps × bigger ILP)
4096 × 14336	4.4× (4× more parallelism per block)
14336 × 4096	6.3× (K/4 + 4× more warps)

> **Checkpoint 5.5** - In 3c, how many threads per block? Per warp? How many warps participate in each output element? - Why does the multi-warp block help even when the kernel was *already winning* at one shape (MLP up/gate)? - The 3c improvement over 3b is "uniform" across shapes (5.4× / 4.4× / 6.3×). What does that uniformity tell us about the *kind* of changes 3c made?

5.6 Why K-split worked for W4A16 (and failed for attention)

In Phase 1 v4 we tried split-K for *attention*, and it lost (§3.6). Here in Phase 3c we use K-split-across-warps for *GEMM*, and it wins big. What's different?

The key difference is **the bottleneck of the previous version**:

Kernel	Previous bottleneck	What K-split does	Result
		Adds blocks; doesn't shorten chain	Loss

Kernel	Previous bottleneck	What K-split does	Result
Phase 1 v3 → v4 attention	Chain-bound (warp reduce + softmax + FMA)		
Phase 3 3b → 3c GEMM	Mixed (grid undersized at small N AND chain long at large K)	Adds parallelism AND splits per-thread K	Win

For attention v3, the chain through `K load → warp_reduce_sum → softmax → V FMA` was the ceiling. Adding more blocks via split-K didn't shorten that chain — every block still had the same per-iter chain. So more SMs busy didn't help.

For GEMM 3b, the chain was *just* the K-FMA loop: `weight load → unpack → FMA → next`. K-split-across-warps directly shortens this chain (each thread does K/4 instead of K iterations). And the multi-warp block gives more warps per SM. Both bottlenecks of 3b are addressed.

The general principle: a pattern's success depends on *whether it attacks the actual bottleneck*. Same pattern, different result, based on what the kernel's chain looks like.

> **Lesson 5.A**: pattern-recognise the bottleneck before applying a textbook optimization. "Split-K" is a hammer; if your nail is a sync barrier, the hammer doesn't help.

5.7 Weight cache and the L2 story

3c at MLP up/gate hits **1577 GB/s achieved weight bandwidth** — more than HBM peak (1008 GB/s). How is that possible?

L2 cache. The 4090's L2 is 72 MiB. Packed W4A16 weights for MLP up/gate are 28.88 MiB — comfortably fitting in L2 once.

On the *first* call, weights load from HBM into L2 (memory-bound on HBM). On every *subsequent* call, weights come from L2 (much higher effective bandwidth). The bench script ran 100 timed iterations after 25 warmups; by iter 5 or so, weights are warm in L2.

This is realistic: in production decode, the same weights get read hundreds of times per request (once per layer × ~200-500 generated tokens). The warm-cache regime is what serving actually sees.

> **Checkpoint 5.7** - The achieved bandwidth of 1577 GB/s for 28.88 MiB weights with 0.018 ms latency works out to "1577 GB/s effective." Is that an HBM peak measurement or an L2 measurement? Why? - In a realistic chat-serving workload (e.g., 8 users × 500 tokens per response × 32 layers), how many times does each weight tensor get read? Why does this matter for cache analysis?

5.8 Phase 3 lessons

1. **The memory-bound thesis transfers across kernel families.** Attention's KV cache (Phase 2), GEMM weights (Phase 3) — same idea, different tensors. When the bottleneck is HBM traffic on a large read-once-per-iter tensor, fewer bytes = potentially less time.
2. **K-split across warps works for GEMM where it didn't for attention.** Different bottleneck patterns. Always pattern- recognise the *current* kernel's bottleneck before applying a textbook trick.
3. **Shmem-cache per-block reuse.** Phase 2c K scales (per group), Phase 3c activations (per block). Same idea: identify the data that's reused many times within a block, lift it out of HBM.

Section B — The cross-cutting workflow

Now we have all the pieces. When you sit down with a new GPU kernel — something you've never seen before — what do you do?

5.9 Step 1: Measure the baseline

Before *any* optimization, get the numbers. Three measurements:

1. **Latency**: CUDA-event-timed, 25 warmup + 100 timed iterations, report median. The `benchmark()` helper in this repo handles it.
2. **Achieved bandwidth**: `bytes_moved / latency`. Compare to:
   - HBM peak (1008 GB/s on 4090)
   - L2 peak (~5 TB/s effective)
   - Useful for memory-bound kernels.

3. **Achieved compute (FLOPS)**: `ops / latency`. Compare to:
    - fp16 Tensor Core peak (~330 TFLOPS on 4090)
    - fp32 CUDA core peak (~83 TFLOPS on 4090)
    - Useful for compute-bound kernels.

The numbers are useful even without a target — they tell you where the time is going.

5.10 Step 2: Categorize — bandwidth, compute, or chain?

The diagnostic tree:

```
Is achieved bandwidth close to HBM peak (or L2 peak)?
  ├── YES → bandwidth-bound. Levers: smaller bytes, better
caching.
  └── NO → continue.

Is achieved compute close to peak FLOPS?
  ├── YES → compute-bound. Levers: Tensor Cores, more
  │          parallelism, fewer ops per result.
  └── NO → continue.

Neither at peak → CHAIN-BOUND. Levers: shorten the inner-loop
chain.
  This is the most common case in our Phase 1-3 kernels.
```

The cheap sanity check: **“if we doubled HBM bandwidth (or doubled compute), would the kernel run twice as fast?”** If not, it’s chain-bound — the per-iter dependency chain is gating throughput, not the resources.

5.11 Step 3: Pick the lever — based on the category

If bandwidth-bound: - Quantize (fewer bytes per element). - Better caching (shmem, L1). - Eliminate redundant reads. - Vectorize loads (fewer load instructions, same bytes).

Phase 2b INT8 KV cache, Phase 3 W4A16 weights both target this.

If compute-bound: - Switch to Tensor Cores (fp16 MMA is ~4× CUDA core FLOPS). - Reduce ops per result (algorithm change). - Vectorize within a warp.

We didn’t hit this in Phases 1-3 (all our kernels were memory or chain bound), but it’s the regime for prefill attention with long prompts.

If chain-bound: - Shorten the inner-loop critical path. Specific tools: - Remove a `__syncthreads()` if you can preserve correctness. - Move a load OUT of the inner loop (per-group pre-compute). - Reshape a serial reduction to use warp shuffle instead of shmem. - Reduce the cycle count of the chain ops (e.g., fewer `expf`).

Phase 1 v2 (single-sync reduce), Phase 1 v3 (single-warp block), Phase 2c (per-group K scales), Phase 3c (K-split across warps) — all target this.

5.12 Step 4: Predict before you measure

This is the discipline that separates "engineer" from "tinkerer": *before* implementing an optimization, predict its effect.

Two predictions:

1. **Direction**: should this make the kernel faster, slower, or neutral?
2. **Magnitude**: roughly how much? (If the prediction is off by 4×, you're missing something.)

The prediction forces you to articulate *why* the optimization helps. If you can't predict, you don't understand the bottleneck.

Common prediction failures from Phases 1-3:

- **Phase 1 v1 naive**: "online softmax = less work = faster." Wrong direction. The diagnosis was incomplete (missed the load-barrier consequence).
- **Phase 1 v4**: "more SMs = more throughput." Wrong magnitude. The prediction assumed bandwidth-bound; actually chain-bound.
- **Phase 2b INT8 KV**: "half the bytes = ~2× faster." Wrong magnitude (was 1×). Same chain-bound diagnosis missed.

Each failure was followed by re-diagnosis: *what's the actual bottleneck?* And the next attempt got it right.

5.13 Step 5: Measure, reconcile, iterate

After implementing:

1. Re-measure all three (latency, bandwidth, compute).
2. Compare to prediction.
3. If on target: great, document and commit.
4. If off: re-diagnose. What did the prediction assume that turned out wrong?

This is the **explore-then-revert pattern** from the project workflow: if the optimization didn't help, commit it to a branch with the diagnostic write-up (so the lesson is captured), then revert main to the previous version. v4 and v5 of attention are examples — both kept in git history at branches `v4` and `v5`, neither on main.

The diagnostic write-up is more valuable than the code. The next time you face a similar shape, you've already paid for the diagnosis.

5.14 Step 6: Pattern-recognise the traps

Four traps that recur (each tied to a Part 3/4 case):

Trap A — "Redundant work is wasteful."

SIMT-parallel ALU work is essentially free (§2.4). Lanes in a warp execute the same instruction in lockstep. If 31 of them would otherwise be idle at a sync, having them do *identical* scalar work costs nothing in wall-clock.

Set-up: you see code where every thread computes the same scalar. Reaction: "wasteful." Truth: probably free.

Trap B — "More SMs = more throughput."

Only if the bottleneck is per-SM compute or per-SM bandwidth. If *total* HBM bandwidth is the ceiling, distributing more SMs to read from it doesn't help. Same for chain-bound kernels — more SMs means each SM still has the same inner-loop chain.

Set-up: kernel runs on few SMs; "let's split-K to fill the GPU." Reaction: check whether SM count is *actually* the bottleneck.

Trap C — "More async loads = more latency hiding."

`cp.async` and similar give you explicit control over in-flight loads. But they go through shmem (the extra hop costs cycles). And nvcc's compiler scheduling already inserts prefetches where it can. The win is real only when: - Per-iter compute is heavy enough to amortize the shmem hop, AND - The current load latency is genuinely on the critical path.

Set-up: latency analysis shows lots of "memory stall" cycles. Reaction: those stalls might already be hidden by occupancy. Don't add shmem hops if it's not helping.

Trap D — "Smaller bytes = faster kernel."

Only if the kernel is bandwidth-bound. Phase 2b INT8 KV measures this exactly: 2× less bytes, 1× speed (tied). The bytes weren't the bottleneck.

The opposite *can* be true: smaller bytes can let you fit more in L2/ L1, which speeds up the warm-cache regime. Phase 3 W4A16 wins partly because 28 MiB packed weights fit in 72 MiB L2 where 112 MiB fp16 don't.

Set-up: "Quantize → smaller bytes → must be faster." Reaction: depends on the bottleneck.

5.15 The named patterns (catalog)

Five patterns from Phases 1-3 that you can reach for, with the condition when each applies:

Pattern 1: Scale-folding linearity (§4.6)

When: a scalar scale appears inside a sum.

What: factor the scalar out: `Σ x · (y · s) = s · Σ x · y`.

Where: INT8 KV attention (`q · k = k_s · q · k_int`); W4A16 (the scale at the *output column* level, after per-channel pre-scaling of q).

Pattern 2: Per-group pre-compute (§4.8)

When: a value is constant within a group of iters but changes between groups.

What: lift the computation involving it to a per-group preamble.

Where: INT4 KIVI per-channel K scales (pre-scale q once per group of seqlen positions); W4A16 group-wise scales (similar structure).

Pattern 3: Single-warp block trade (§3.5)

When: cross-warp sync is on the inner-loop critical path AND you can afford to drop occupancy.

What: shrink to 1 warp per block, eliminate `__syncthreads()`, use warp shuffle for the cross-thread communication.

Where: Phase 1 v3, all Phase 2 attention kernels.

Pattern 4: Double-buffered shmem reduce (§3.4)

When: a cross-warp shmem reduce per iter has 2 syncs (one for the write, one for the broadcast).

What: alternate buffers iter-to-iter so iter j+1's write can't clobber iter j's read. Pair with "every warp redundantly does the final reduce" so the broadcast is via shuffle, not shmem.

Where: Phase 1 v2.

Pattern 5: K-split across warps (§5.6)

When: kernel is chain-bound on a long K-reduction AND grid is too small to fill the GPU.

What: multi-warp block, K split across warps, tiny shmem combine for the per-column partials.

Where: Phase 3c (worked); Phase 1 v4 attempted (didn't work because attention's chain wasn't K-reduction-bound).

5.16 The six-question checklist

When you sit down with a new GPU kernel:

1. **What's the workload?** Shape of inputs, expected output, target workload (decode shape vs prefill shape, etc.).
2. **What does the kernel look like at the inner loop?** What are the per-iter loads, the per-iter syncs, the per-iter critical path?
3. **What's the baseline?** Latency + achieved bandwidth + achieved compute. Compared to peak HBM, peak compute.
4. **Bandwidth-, compute-, or chain-bound?** Use §5.10's diagnostic tree.
5. **What lever shortens *that* specifically?** Match the category to §5.11's list of levers. Predict the magnitude.
6. **Measure, reconcile, iterate.** If the prediction was wrong, re-diagnose.

This is the working playbook. Every kernel in Phases 1-3 was developed by applying these six steps repeatedly. Most failures came from skipping step 4 (incorrect categorization) or step 5 (wrong lever for the actual bottleneck).

5.17 Closing

We're 23,000 words and five parts in. The book started from "what does attention compute?" and ended at "what do you do when you sit down with a new kernel?" Along the way:

- **Part 1**: the math — Q, K, V, softmax, GQA, prefill vs decode, the KV cache.
- **Part 2**: the hardware — SMs, warps, threads, memory hierarchy, SIMT, occupancy, the dependency chain, the diagnostic framework.
- **Part 3**: decode attention v0 → v5 — five wins and two reverts, with the bottleneck and lesson articulated at each step.
- **Part 4**: KV-cache compression — quantization math, KIVI's K vs V asymmetry, the scale-folding linearity trick, the per-group pre-compute trick, the kernel-level vs model-level gap.
- **Part 5**: W4A16 GEMM and the cross-cutting playbook — multi-warp K-split, the four recurring traps, the five named patterns, the six-question checklist.

The five lessons one more time

1. **SIMT-parallel ALU is essentially free.** Don't move work to one lane to "save" it if the other lanes were idle.

2. **`__syncthreads()` is a load barrier.** nvcc won't hoist memory loads above it. Manual prefetching matters.

3. **Removing a shmem hop can dwarf removing a sync.** The full consequences of shortening the chain often exceed the sync's own cost.

4. **Occupancy is a means, not an end.** Trade it for shorter chain when the lost warps were idle at syncs.

5. **Don't add parallelism where the bottleneck isn't compute.** Bandwidth-bound parallelism through more SMs doesn't help if HBM is already the shared ceiling.

(Sub-lessons from Phase 2 and Phase 3 are absorbed into the named patterns and the trap list.)

What to do next

You're ready to:

1. **Re-implement v0 → v3 from memory.** Check out branch `v0`, run the tests, study the code, write your own version, validate against the reference.

2. **Walk new kernels using the checklist.** Anything in the `flash-attn` library, `cublas`, or your favorite GPU codebase. Apply §5.16.

3. **Teach it.** Find someone curious and explain Phase 1 v2's single-sync reduce. If you can articulate *why* it wins more than the visible sync removal predicts, you've got the chain-bound mental model.

4. **Read the journey docs** (`docs/0[123]-*-journey.md`) again. The first time they were narrative; the second time they're applied exercises in this part's framework.

The repo's branches are your study path. `v0` through `v5`, `2a` through `2d`, `3a` through `3c` — each branch is a self-contained state you can build, test, and benchmark. Use them.

Done. Part 5 is the end of the book.

The framework is yours to take elsewhere — the journey was always the framework, not the specific kernels.